Cairo Higher Institute for Engineering, Computer Science and Management

معهد القاهره العالي للهندسه وعلوم الحاسب والاداره

System Analysis and Design Project

Text Book Inventory

Supervisor: Eng. Abd El-Rahman Samy

Team Leader: Ahmed Shawky

ID: 2018030020

Team Member 1: Bassant Mohamed

ID: 2018030047

Team Member 2: Youstina Nashaat

ID: 2018030269

Team Member 3: Omar Allam

ID: 2018030086

Cairo 2020

Abstract

Design and construction of a

Design and construction of text book inventory, in this paper, we designed and implemented a system to easily check and manage book inventory using Desktop Application. In the previous conventional method, the user visits the bookstore directly and then checks the inventory. However, in this case, the user accessibility is inconvenient, and it is troublesome to check or reconfirm the inventory. In order to solve these inconveniences, we provide book inventory management system for requesting book purchase and inquiring bulletin board by book field, and then providing real time inventory check with book application and purchase functions. In addition, through the push notification function, we can catch up with the purchase status of the ordered books on Desktop Application in real time. Therefore, we designed and implemented a function that enables bookstore operators and users to easily and conveniently manage by providing statistical information on inventory data.

Acknowledgment

First of all, we have to be thanks for each of:

- Dr. Osama of being as a good father with us and supporting us not out team only but all of my friends and colleagues' thanks for him.
- Eng. Abdelrahman Samy of being a big brother and helper and he helps us a lot in diagrams.
- Last but not least Eng. Nisreen as she made her effort to facilitate and deliver the subject as more possible as she could.
- And about us, we have to thank each other for being supporter to each other and helpful one to make that project.

List of Figures

Table of Contents

Appendix

Chapter One
Introduction and Purpose

1.1 Introduction

In this chapter we will give a small definition about the project and talk about an overview of the project. This Chapter is divided into six Sections: - Section 1.1 represent the introduction of the Chapter, Section 1.2 list of basic definition, Section 1.3 over view about project, Section 1.4 show the problem definition, Section 1.5 represent the project objectives and finally Section 1.6 concludes this chapter.

1.2 Basic Definition:

inventory management is a systematic approach to sourcing, storing, and selling inventory—both raw materials (components) and finished goods (products).

In business terms, inventory management means the right stock, at the right levels, in the right place, at the right time, and at the right cost as well as price.(1)

1.3 Over View:

Over all this platform help owners to manage their employees and knows all information about them, and also help them to know about their stock and check if there is a problem with it to find a solution for that in a short time and also help the user to view and search for any book in the system without asking the employees or the sellers, and when we mentioned the sellers the system also help them to

print bills quicker than before and search for any book they want for any customer and add or delete any book in the system and check if there is any wrong numbers of the available books to check it with the admin (manager).

1.4 Problem Definition:

In a continuous review system, managers continuously monitor the inventory position. Whenever the inventory position falls at or below a level R, called the reorder point, the manager orders Q units, called the order quantity. (Notice that the reorder decision is based on the inventory position including orders and not the inventory level. If managers used the inventory level, they would place orders continuously as the inventory level fell below R until they received the order.) When you receive the order after the lead-time, the inventory level jumps from zero to Q, and the cycle repeats.

In inventory systems, demand is usually uncertain, and the lead-time can also vary. To avoid shortages, managers often maintain a safety stock. In such situations, it is not clear what order quantities and reorder points will minimize expected total inventory cost. Simulation models can address this question. (2)

1.5 Project Objectives:

At this point we will discuss the actors and what there objectives:

Admin:

Add, Delete, search and edit books.

Add, delete, search employees.

Login and logout as admin.

Print reports.

View home page.

view and search for any item in the system.

Seller:

Add, delete and search book.

View profile.

Print bills.

View home page.

Login, logout to the system as seller.

User:

View profile.

Search for any book in the system to check if it available or not.

1.6 References

(1) https://www.tradegecko.com/inventory-management

(2) https://docs.oracle.com/cd/E57185_01/CBREG/ch07s01s10s01.html

1.7 Conclusion:

At the end of this chapter we discussed the overall information of the system about the basics like the basic definition and the problem definition and how to fix it and what every actor in the system do and who has the permission to enter the data and who is not.

Chapter Two
Overview about Project Name

2.1 Introduction:

In this chapter we will discuss the desktop system that we did in our project. This chapter has 6 sections starting with introduction and ending with related projects. We will take about its publisher and history of it .

2.2 History of project:

Desktop System , which is know as Open Solaris Desktop December 1990 and this name was related to the first company who developed a desktop system (Sun Microsystem) and then it was developed and upgraded by Oracle .

Desktop System aims to provide a system familiar to the average computer user and make most of operation easily as calculating , calendaring , browsing , etc……..

You can do any Desktop Systems by variable Languages like C , C++ and Java and each Language has many operating systems can be Written on it .

In this topic we are going to take you a round Desktop System by Java Language because this Language what we use in our Project .

Java is a set of computer software and specifications developed by Sun Microsystems, which was later acquired by the Oracle Corporation that provides a system for developing application software.

2.3 References :

- Wikipedia
- Oracle home page

Chapter Three
System Analysis

3.1 Introduction

In this chapter we analysis and design the system by introduce the algorithm We are used in the project; insection3.2 we will explain the system of helicopter camera which include FD and the discussion of it, DFD and the discussion of it .

3.2 System Analysis

3.2.1 System Functional Diagram (FD)

We introduce in this section FD for the system and discuss of it Function diagram is a process of divisions from a higher function Appropriate smaller function which shown in the figure 3.1

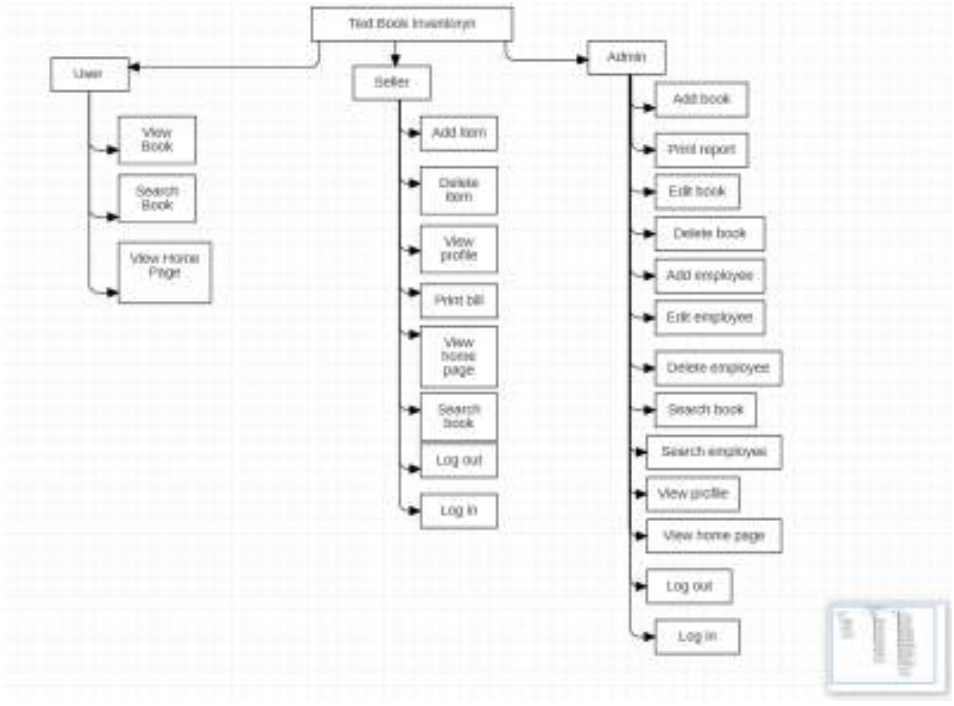


Figure 3.1 Functional Diagram

3.2.2 Context Diagram

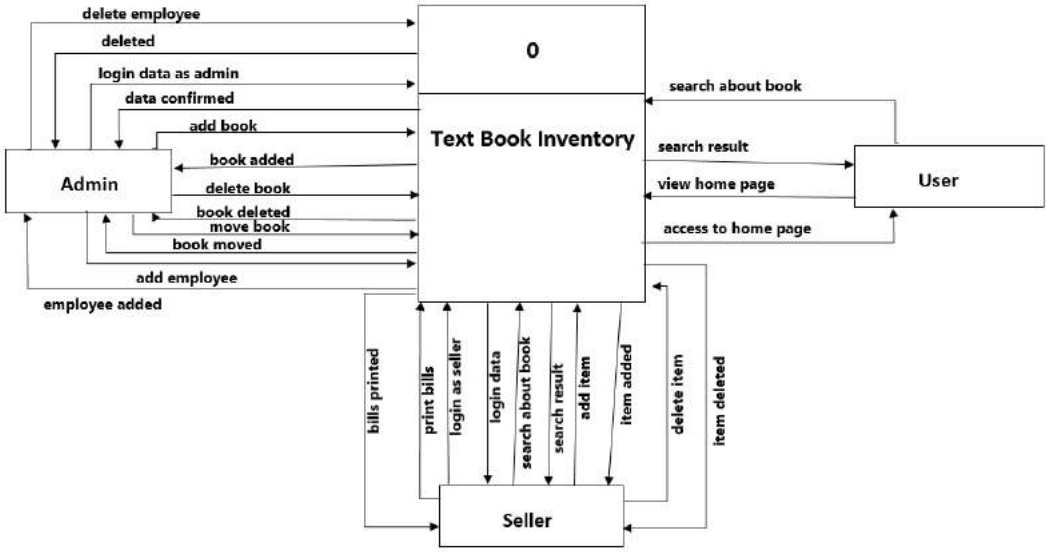


Figure 3.2 Context Diagram

A Context Diagram (CD) in software engineering and systems engineering is a diagram that defines the boundary between the system, or part of a system, and its environment, showing the entities that interact with it.This diagram is a high level view of a system. [1]

3.2.3 Data Flow Diagram (DFD)

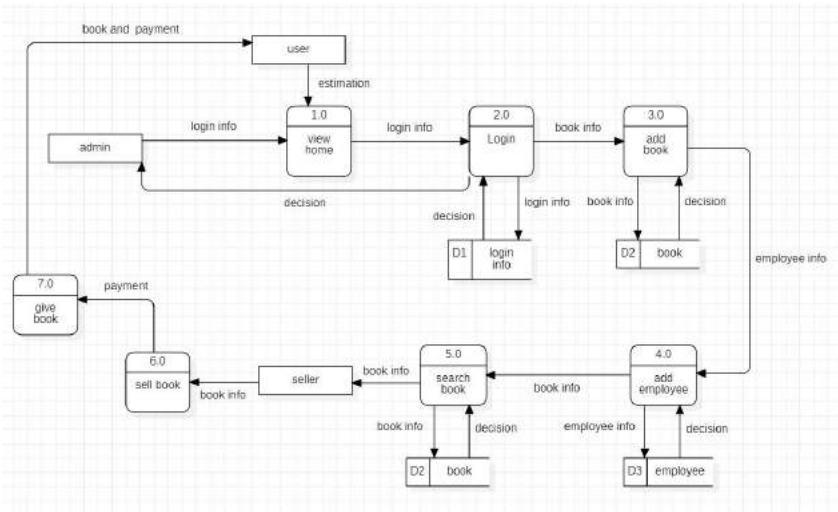


Figure 3.3 Data flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the "flow" of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design). [1]

3.2.4 Represents DFD Data flow diagram: Level 1

This level one data flow diagram template can help you:

- Map out the flow of information for any process/system.
- Visualize a high-level overview of the whole system/process.
- Share an overview with stakeholders and developers.

Open this template to view a level one data flow diagram example that you can customize to your use case.

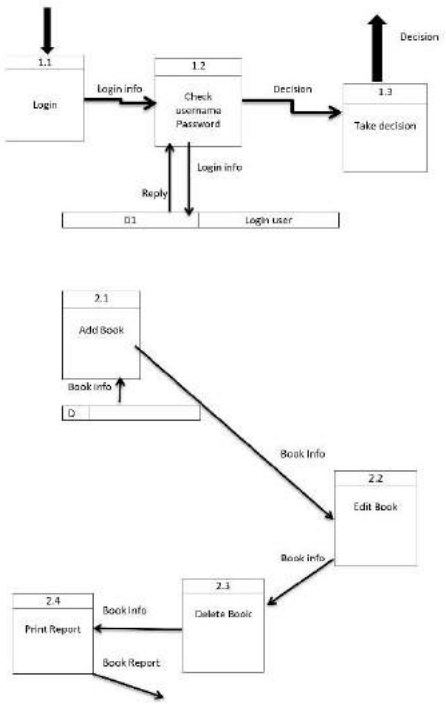


Figure 3.4 DFD L1

3.2.5 Sequence Diagram

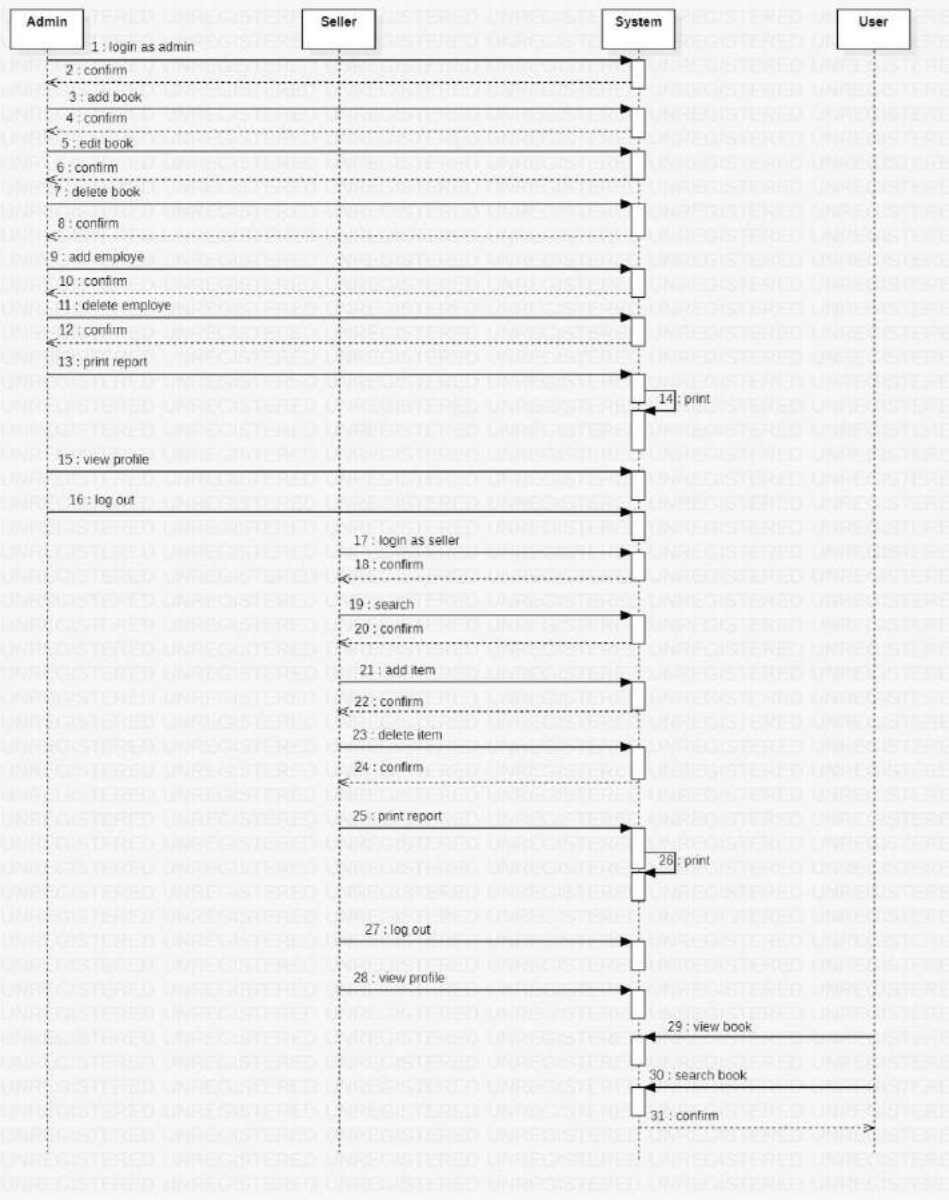


Figure 3.5 Sequence Diagram

A sequence diagram shows, as parallel vertical lines (lifelines), different processes or objects that live simultaneously, and, as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner. [1]

3.2.6 use-case Diagram

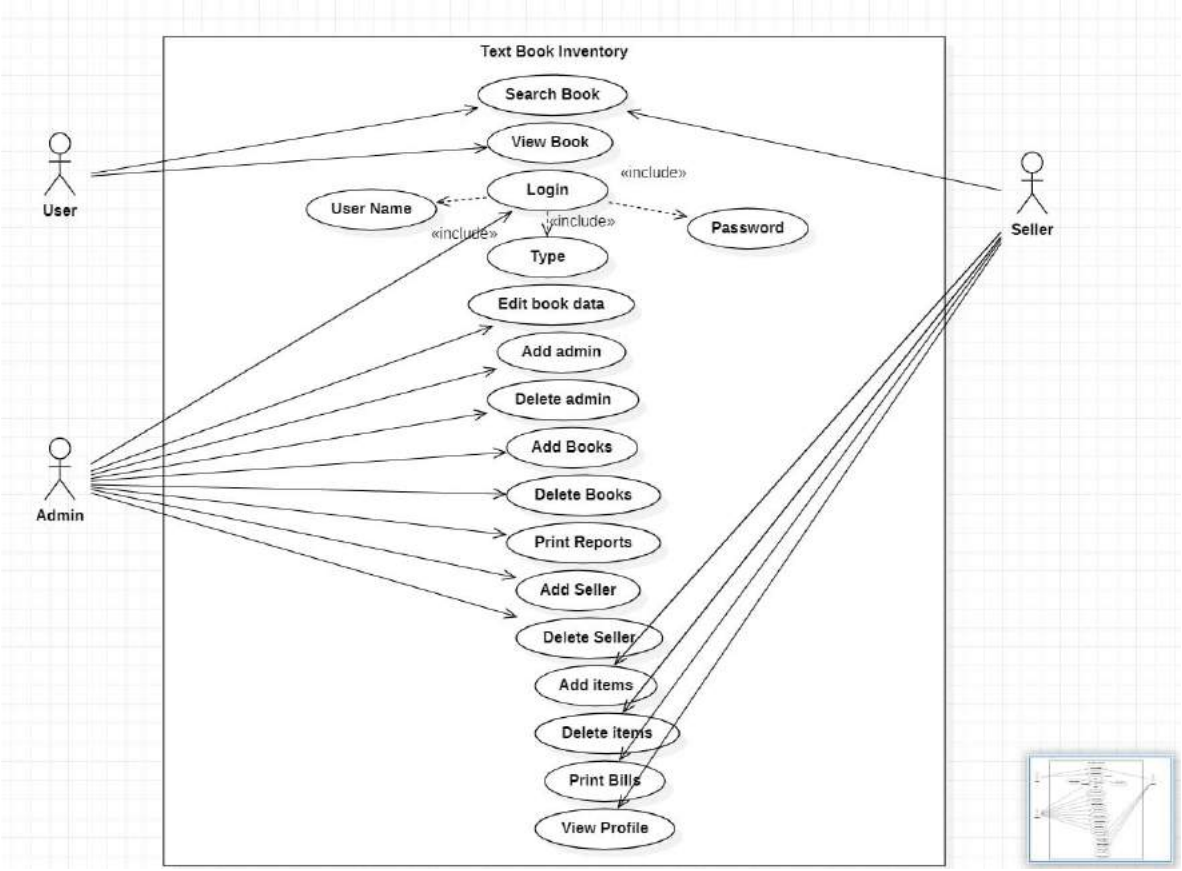


Figure 3.6 use-case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. A use case diagram can identify the different types of users of a system and the different use cases and will often be accompanied by other types of diagrams as well. [1]

3.2.7 Class and Object Diagram

Class diagram is UML structure diagram which shows structure of the designed system at the level of classes and interfaces, shows their features, constraints and relationships - associations, generalizations, dependencies, etc.

Some common types of class diagrams are:

domain model diagram,

diagram of implementation classes.

Object diagram could be considered as instance level class diagram which shows instance specifications of classes and interfaces (objects), slots with value specifications, and links (instances of association)

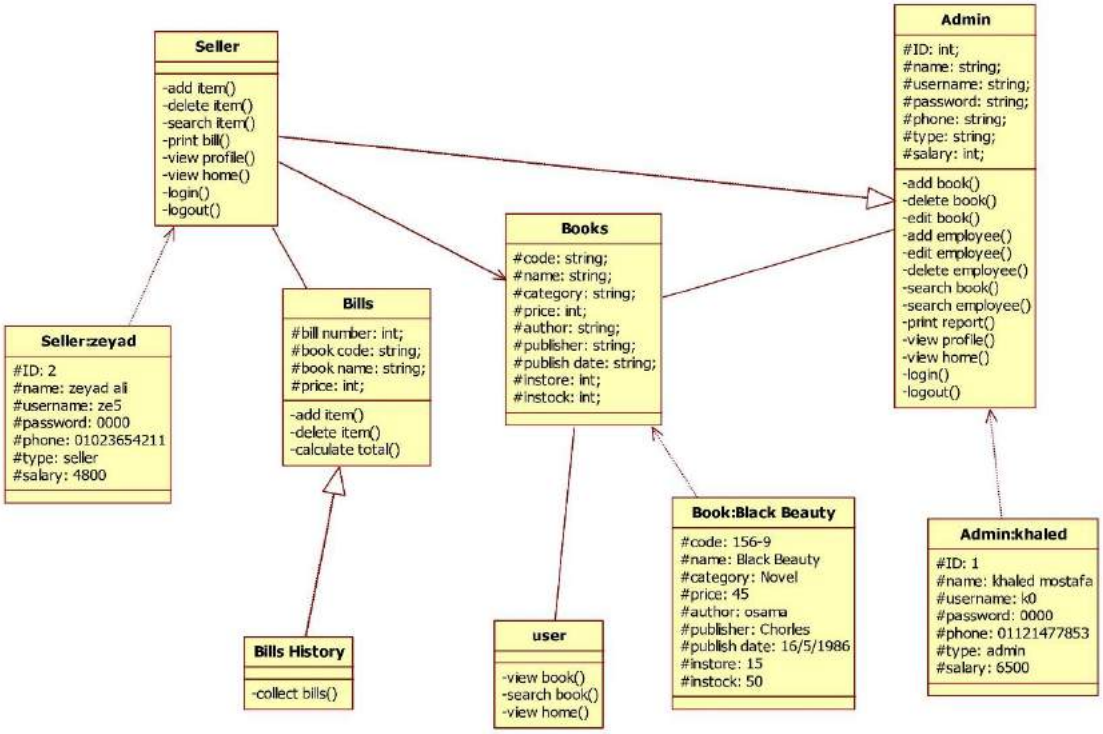
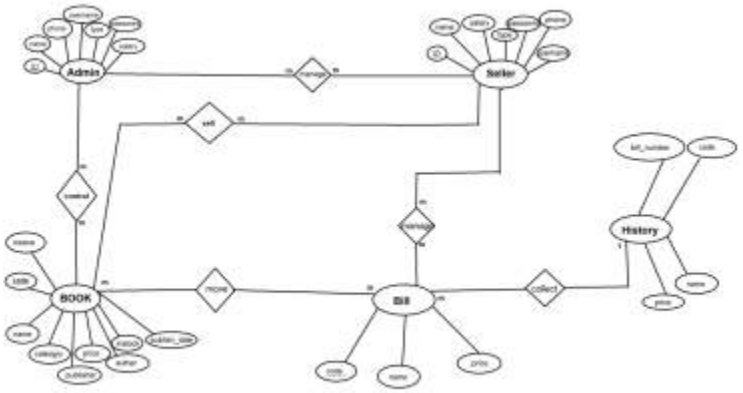


Figure 3.7 Class & Object Diagram

3.2.8 ER diagram:

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how “entities” such as people, objects or concepts relate to each other within a system. ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research. Also known as ERDs or ER Models, they use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes. They mirror grammatical structure, with entities as nouns



and relationships as verbs.

Figure 3.8 ERD Diagram

3.2.9 state diagram :

A state diagram is a type of diagram used in computer science and related fields to describe the behavior of systems. State diagrams require that the system described is composed of a finite number of

states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics.

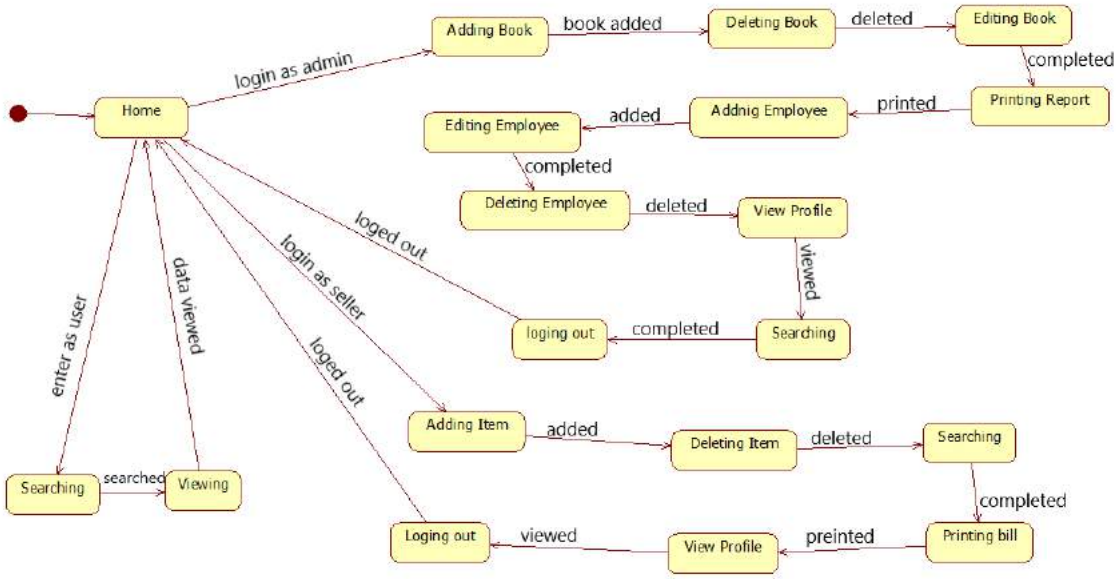


Figure 3.9 State Diagram

3.2.10 Activity diagram :

Activity diagram is another important behavioral diagram in UML diagram to describe dynamic aspects of the system. Activity diagram is essentially an advanced version of flow chart that modeling the flow from one activity to another activity.

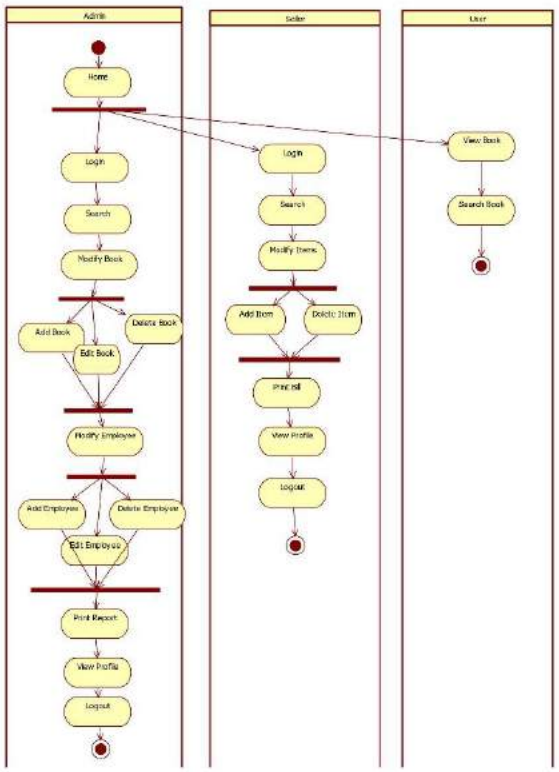


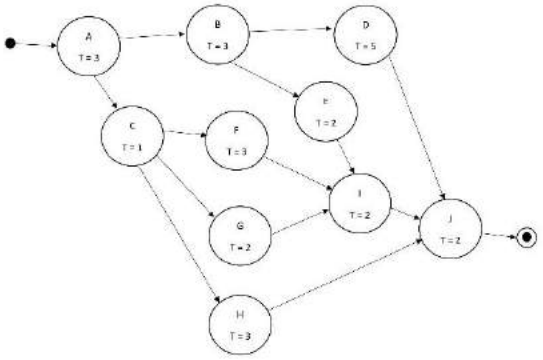
Figure 3.10 Activity Diagram

3.2.11 PERT& Gant :

A PERT chart is a visual project management tool used to map out and track the tasks and timelines. The name PERT is an acronym for Project (or Program) Evaluation and Review Technique.

	Activity	Order	Dependencies
A	Planning	None	3
B	Database Design	A	3
C	Module logout	A	1
D	Database computer	B	5
E	Database interface	B	2
F	Input Module	C	3
G	Output Module	C	2
H	GUI Structure	C	3
I	I/O interface implementation	E , F , G	2
J	Final Testing	D , H , I	2

Activity on Node (AON) :-



Activity on Arrow (AOA): -

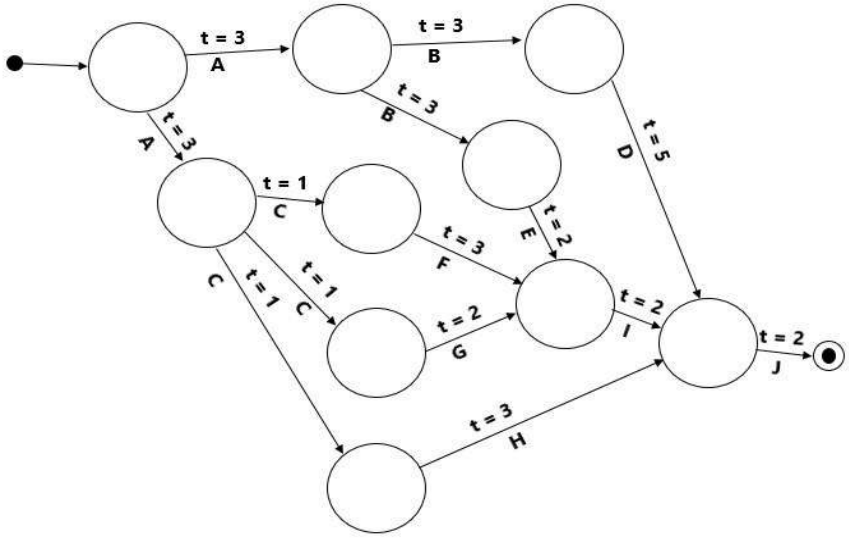


Figure 3.11 :Pert Diagram

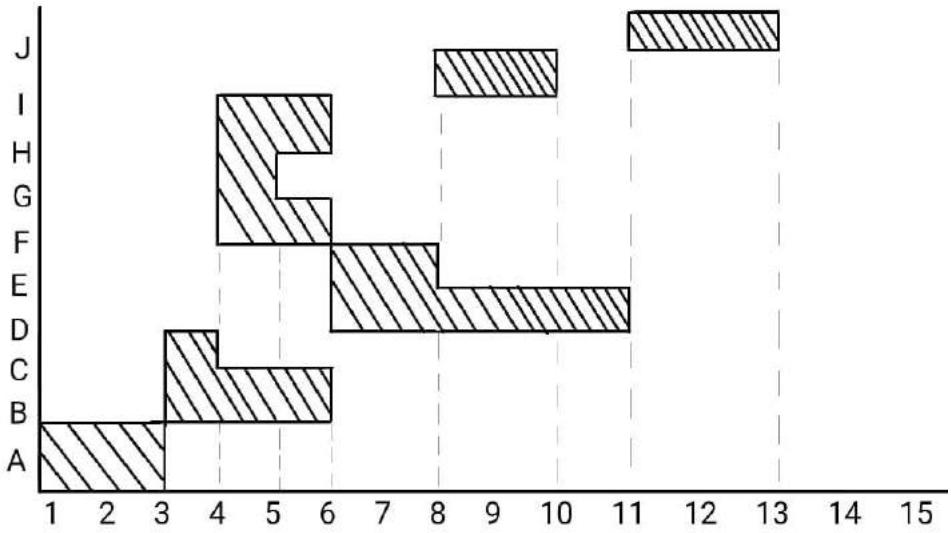


Figure 3.12 Gantt Diagram

3.3 references

- https://www.google.com/search?q=Functional+diagram+define&source=lmns&hl=en&sa=X&ved=2ahUKEwitw-W2kvTtAhVRwYUKHUUSBKIQ_AUoAHoECAEQAA
- https://en.wikipedia.org/wiki/Use_case_diagram
- https://en.wikipedia.org/wiki/State_diagram
- https://www.lucidchart.com/pages/templates/data-flow/lucidchart-data-flow-diagram-level-1
- https://www.uml-diagrams.org/class-diagrams-overview.html
- https://www.lucidchart.com/pages/er-diagrams#section_0
- https://www.modernanalyst.com/Careers/InterviewQuestions/tabid/128/ID/1433/What-is-a-Context-Diagram-and-what-are-the-benefits-of-creating-one.aspx

- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/
- https://www.praxisframework.org/en/library/activity-on-arrow-diagram#:~:text=In%20an%20activity%2Don%2Darrow,called%20the%20i%2Dnode).&text=A%20network%20diagram%20is%20created,their%20dependence%20upon%20each%20other
- https://www.productplan.com/glossary/pert-chart/
- https://www.lucidchart.com/pages/data-flow-diagram#:~:text=DFD%20Level%200%20is%20also,its%20relationship%20to%20external%20entities

3.4 Conclusion

In this chapter we learned how to make system analysis to the project from FD, DFD; we know which algorithm we are used in our project.

Chapter Four
Implementation

4.1 Introduction:

In this chapter presents the implementation steps of the project by taking snapshots to the project, the function of apps and Project.

4.2 software component

NetBeans IDE 8.2

Derby Database (built-in)

4.3 software function

4.3.1 Home Page Functions

4.3.1.1 Login Function to go to Login page

4.3.1.2 Search by name Function for all users

4.3.1.3 Apply Function to View all Books With the same Category

4.3.2 Login Page

4.3.2.1 Login Function for the Admin and The Seller

4.3.2.2 Back Function to return to the Home Page

4.3.3 Admin Page

4.3.3.1 Employee Function to go to Employee Page

4.3.3.2 Books Function to go to Books Page

4.3.3.3 Stock Function to go to Stock Page

4.3.3.4 Profile Function to go to Profile Page

4.3.3.5 Logout Function to return back to Home Page

4.3.4 Employee Page

4.3.4.1 Search Function to search for specific Employee

4.3.4.2 Clear Function to clear all text field for new search

4.3.4.3 Add Employee Function to add new Employee to the system

4.3.4.4 Edit Employee Function to update Employee data

4.3.4.5 Delete Employee Function to Delete an Employee from the system

4.3.4.6 Back Function to return to Admin Page

4.3.5 Books Page

4.3.5.1 Search Function to search for specific Book

4.3.5.2 Clear Function to clear all text field for new search

4.3.5.3 Add Book Function to add new Book to the system

4.3.5.4 Edit Book Function to update Book data

4.3.5.5 Delete Book Function to Delete a Book from the system

4.3.5.6 Back Function to return to Admin Page

4.3.6 Stock Page

4.3.6.1 Search Function to search for specific Book

4.3.6.2 Clear Function to clear text field for new search

4.3.6.3 Add to Store Function to move book from stock to store

4.3.6.4 Print Report Function to print report with all books data in the system

4.3.6.5 Back Function to return to Admin Page

4.3.7 Profile Page

4.3.7.1 Back Function to return to previous Page whether Admin or Seller

4.3.8 Seller Page

4.3.8.1 Search Function to search for specific Book

4.3.8.2 Clear Function to clear text field for new search

4.3.8.3 Add Item Function to add book to the bill

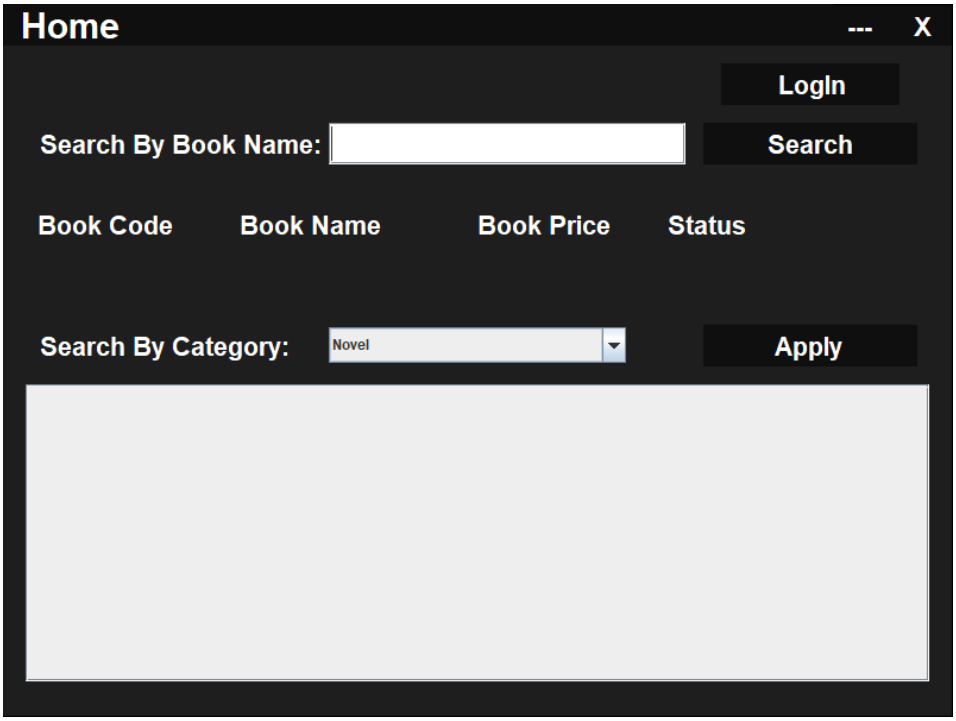
4.3.8.4 Print Bill Function to print the bill

4.3.8.5 Delete Item Function to delete book from the bill

4.3.8.6 Profile Function to go to Profile Page

4.3.8.7 Logout Function to return to Home Page

4.4 software snapshots



4.4.1 Interface Design

Figure [4.1] Home Page

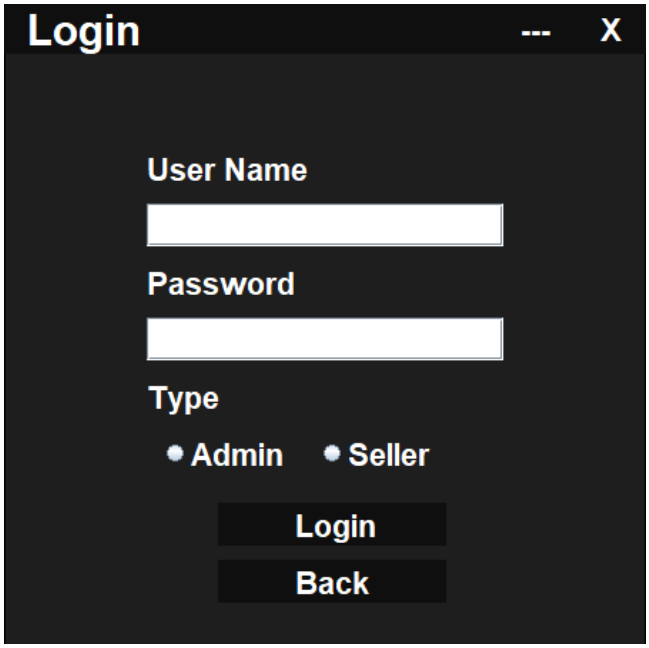


Figure [4.2] Login Page

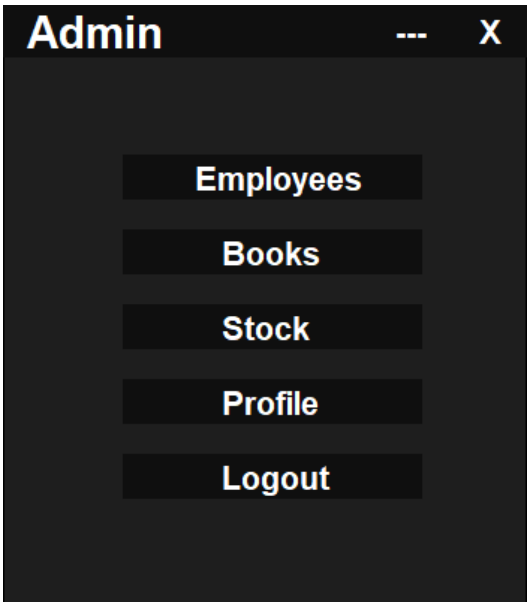


Figure [4.3] Admin Page

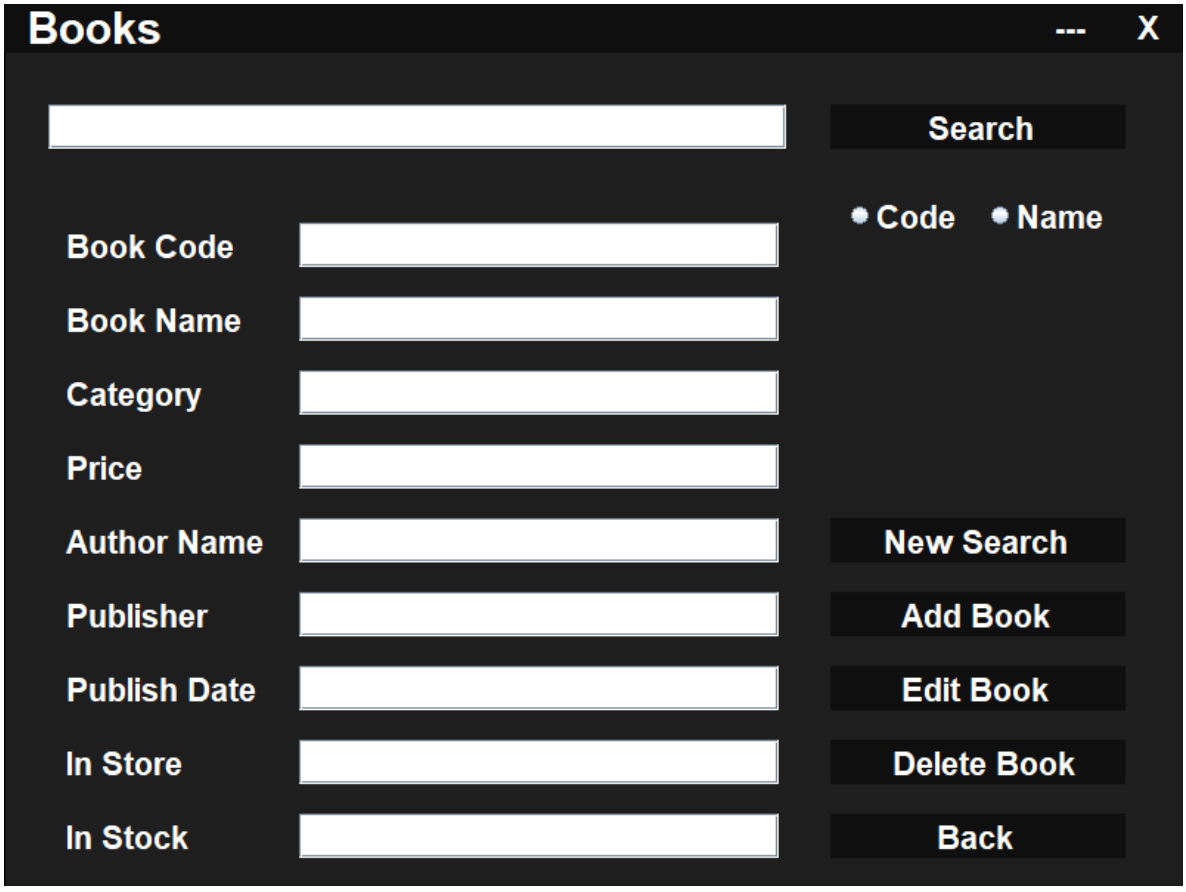
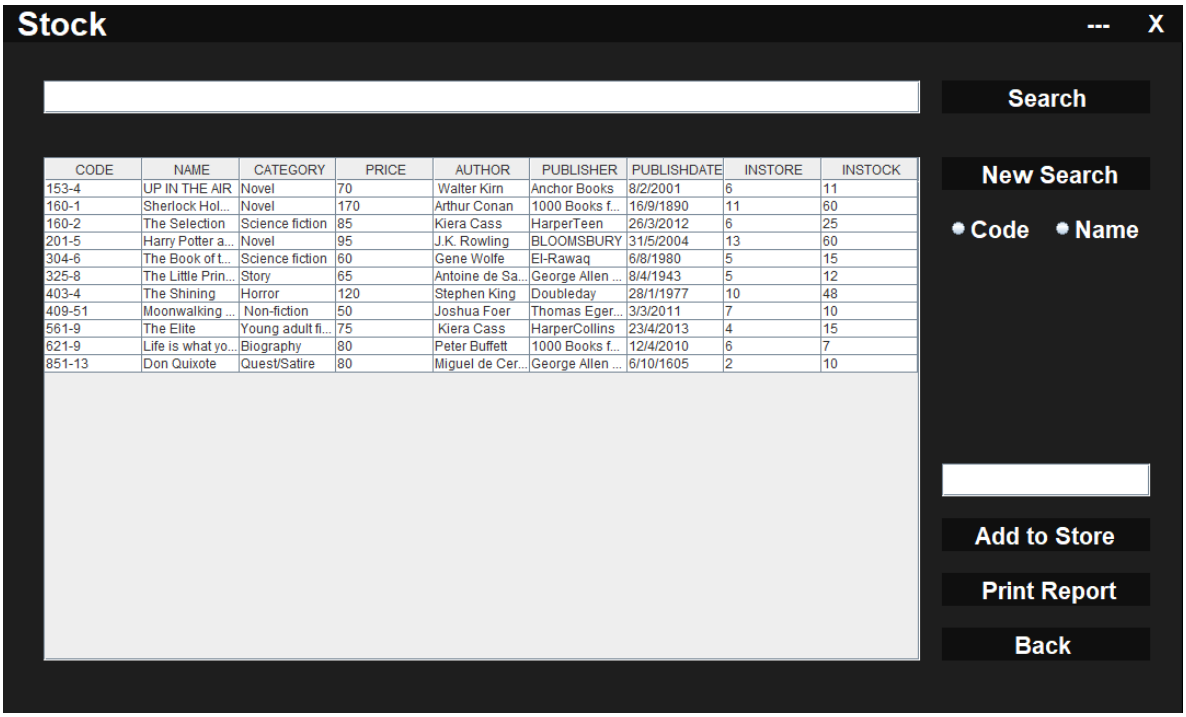


Figure [4.4] Books Page



CODE	NAME	CATEGORY	PRICE	AUTHOR	PUBLISHER	PUBLISHDATE	INSTORE	INSTOCK
153-4	UP IN THE AIR	Novel	70	Walter Kirn	Anchor Books	8/2/2001	6	11
160-1	Sherlock Hol...	Novel	170	Arthur Conan	1000 Books f...	16/9/1890	11	60
160-2	The Selection	Science fiction	85	Kiera Cass	HarperTeen	26/3/2012	6	25
201-5	Harry Potter a...	Novel	95	J.K. Rowling	BLOOMSBURY	31/5/2004	13	60
304-6	The Book of t...	Science fiction	60	Gene Wolfe	El-Rawaq	6/8/1980	5	15
325-8	The Little Prin...	Story	65	Antoine de Sa...	George Allen ...	8/4/1943	5	12
403-4	The Shining	Horror	120	Stephen King	Doubleday	28/1/1977	10	48
409-51	Moonwalking ...	Non-fiction	50	Joshua Foer	Thomas Eger...	3/3/2011	7	10
561-9	The Elite	Young adult fi...	75	Kiera Cass	HarperCollins	23/4/2013	4	15
621-9	Life is what yo...	Biography	80	Peter Buffett	1000 Books f...	12/4/2010	6	7
851-13	Don Quixote	Quest/Satire	80	Miguel de Cer...	George Allen ...	6/10/1605	2	10

Figure [4.5] Stock Page

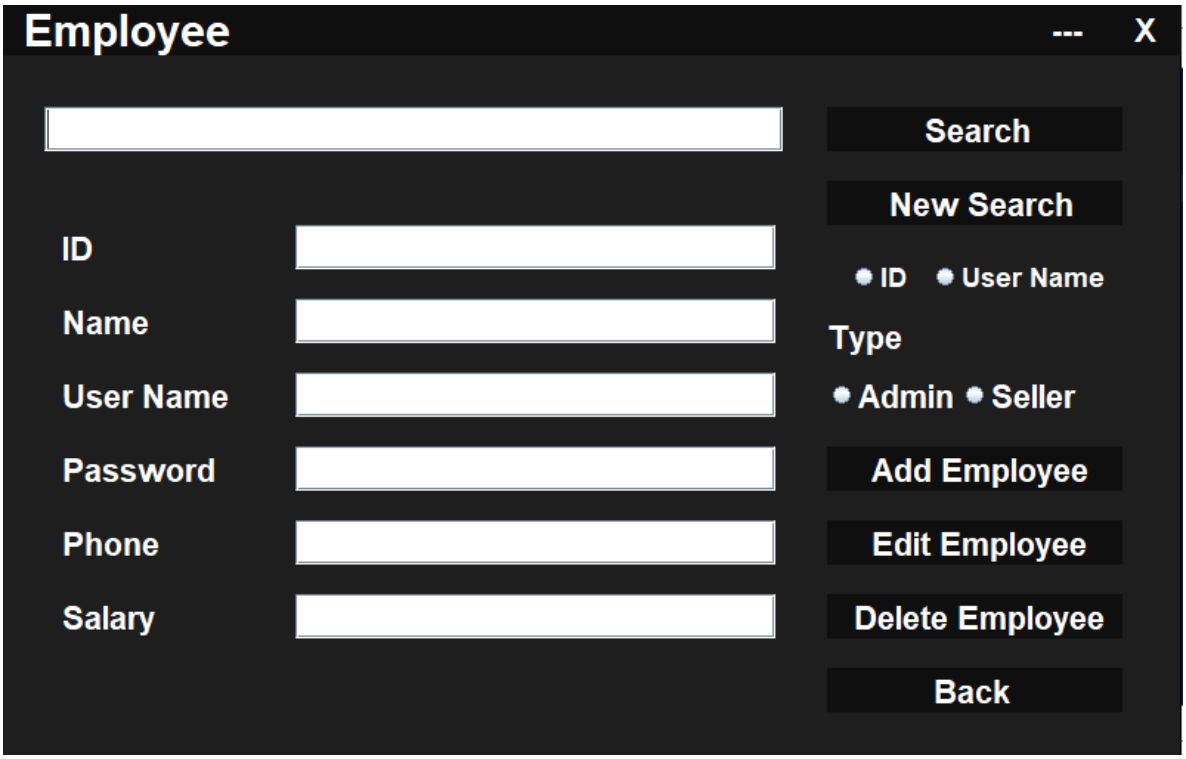


Figure [4.6] Employee Page

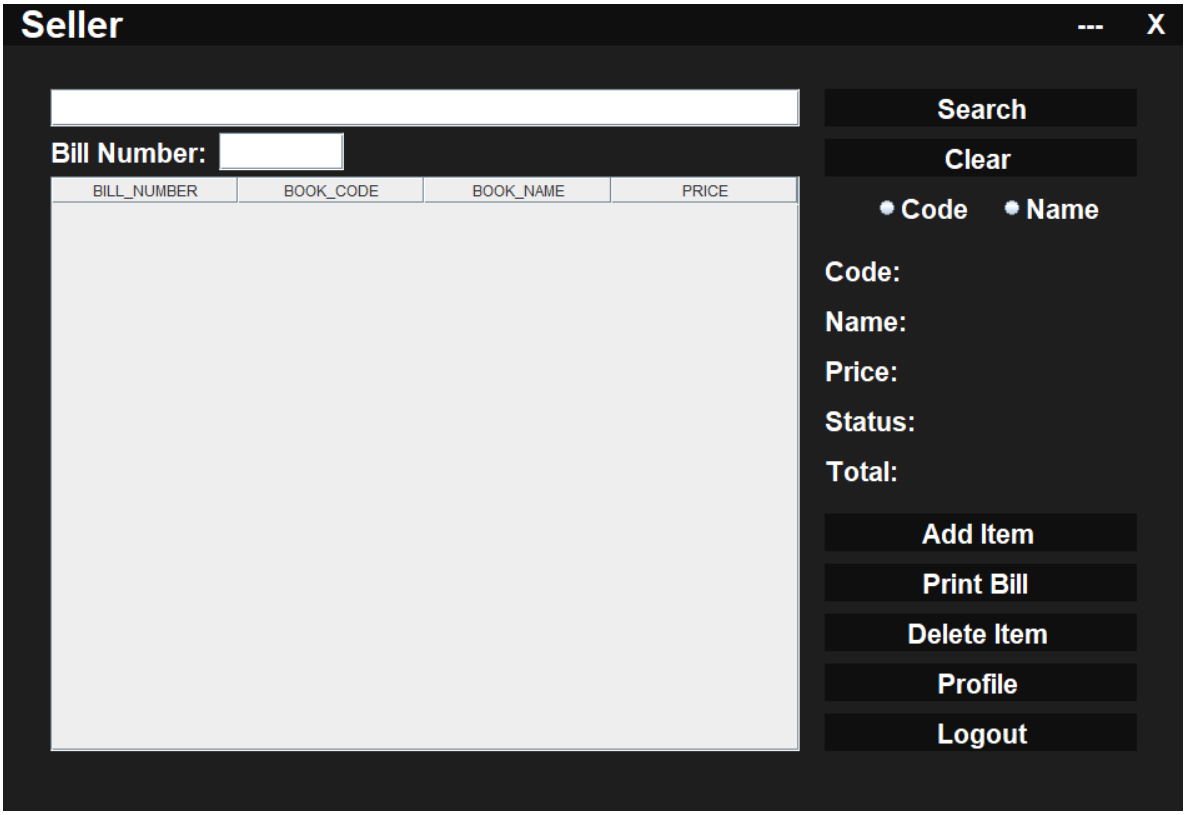


Figure [4.7] Seller Page

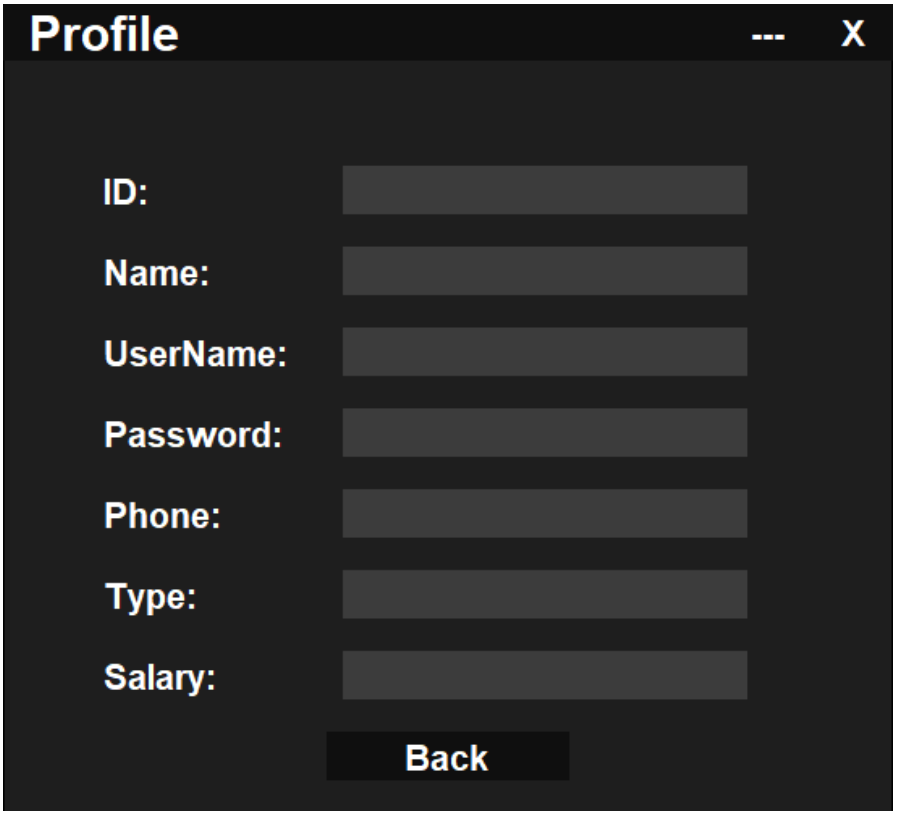


Figure [4.8] Profile Page

4.4.2 Database Design

SELECT * FROM A."ADMIN" F... ×

Max. rows: 100 | Fetched Rows: 2 | Matching Rows:

#	ID	NAME	USERNAME	PASSWORD	PHONE	TYPE	SALARY
1	1	ahmed	a6	2222	01124557869	admin	6400
2	2	osama	x1	1111	01047563210	admin	6200

Figure [4.9] Admin Table

SELECT * FROM A.SELLER FE... ×

Max. rows: 100 | Fetched Rows: 2 | Matching Rows:

#	ID	NAME	USERNAME	PASSWORD	PHONE	TYPE	SALARY
1	1	khaled	k0	0000	01125696984	seller	3200
2	2	islam	is 16	3333	01254584000	seller	4300

Figure [4.10] Seller Table

SELECT * FROM A.BOOKS FET... ×

Max. rows: 100 | Fetched Rows: 11 | Matching Rows:

#	CODE	NAME	CATEGORY	PRICE	AUTHOR	PUBLISHER	PUBLISHDATE	INSTORE	INSTOCK
1	160-1	Sherlock Holmes	Novel	170	Arthur Conan	1600 Books for Publishing	16/9/1890	14	60
2	160-2	The Selection	Science fiction	85	Kiera Cass	HarperTeen	26/3/2012	6	25
3	153-4	UP IN THE AIR	Novel	70	Weber Kim	Anchor Books	8/2/2001	6	11
4	201-5	Harry Potter and the Prisoner of Azkaban	Novel	95	J.K. Rowling	BLOOMSBURY	31/5/2004	13	60
5	304-6	The Book of the New Sun	Science fiction	60	Gene Wolfe	El Rowac	6/8/1980	5	15
6	561-9	The Elite	Young adult fiction	75	Kiera Cass	Harper Collins	28/4/2013	4	15
7	323-8	The Little Prince	Story	65	Antoine de Saint-Exupéry	George Allen & Unwin	8/4/1943	5	12
8	851-13	Don Quixote	Quest/Satire	80	Miguel de Cervantes	George Allen & Unwin	6/10/1605	2	10
9	409-51	Moonwalking with Einstein	Non-fiction	50	Joshua Foer	Thomas Egerton	3/3/2011	7	10
10	621-9	Life is what you make it	Biography	80	Peter Buffett	1600 Books for Publishing	12/4/2010	6	7
11	403-4	The Shining	Horror	120	Stephen King	Doubleday	28/1/1977	11	50

Figure [4.11] Books Table

SELECT * FROM A.BILLS FET... ×

Max. rows: 100 | Fetched Rows: 0 | Matching Rows:

#	BILL_NUMBER	BOOK_CODE	BOOK_NAME	PRICE

Figure [4.12] Bill Table

SELECT * FROM A.HISTORY F... ×

Max. rows: 100 | Fetched Rows: 6 | Matching Rows:

#	BILL_NUMBER	BOOK_CODE	BOOK_NAME	PRICE
1	1	160-2	The Selection	85
2	1	201-5	Harry Potter and the Prisoner of Azkaban	95
3	1	561-9	The Elite	75
4	2	561-9	The Elite	75
5	2	304-6	The Book of the New Sun	60
6	2	160-1	Sherlock Holmes	140

Figure [4.13] History Table

Book Report

CODE	NAME	CATEGORY	PRICE	AUTHOR	PUBLISHER	PUBLISHDATE	INSTORE	INSTOCK
153-4	UP IN THE AIR	Novel	70	Walter Kirn	Anchor Books	8/2/2001	6	11
160-1	Sherlock Hol...	Novel	170	Arthur Conan	1000 Books f...	16/9/1890	14	60
160-2	The Selection	Science fiction	85	Kiera Cass	HarperTeen	26/3/2012	6	25
201-5	Harry Potter a...	Novel	95	J.K. Rowling	BLOOMSBURY	31/5/2004	13	60
304-6	The Book of t...	Science fiction	60	Gene Wolfe	El-Rawaq	6/8/1980	5	15
325-8	The Little Prin...	Story	65	Antoine de Sa...	George Allen ...	8/4/1943	5	12
403-4	The Shining	Horror	120	Stephen King	Doubleday	28/1/1977	11	50
409-51	Moonwalking ...	Non-fiction	50	Joshua Foer	Thomas Eger...	3/3/2011	7	10
561-9	The Elite	Young adult fi...	75	Kiera Cass	HarperCollins	23/4/2013	4	15
621-9	Life is what yo...	Biography	80	Peter Buffett	1000 Books f...	12/4/2010	6	7
851-13	Don Quixote	Quest/Satire	80	Miguel de Cer...	George Allen ...	6/10/1605	2	10

Book Report

Figure [4.14] Book Report

Books Bill

BILL_NUMBER	BOOK_CODE	BOOK_NAME	PRICE
2	160-1	Sherlock Holmes	140
3	304-6	The Book of the New Sun	60
3	561-9	The Elite	75

Total: 275 EGP

Figure [4.15] Bill

Chapter Five
Conclusion and future work

5.1 Introduction.

This chapter will conclude the system; it talk about two section:

- The first section introduce the conclusion of the system.

- The second section will talk about the improvements that will be made in future work.

5.2 Conclusion

The Textbook Inventory Management System (TIMS) is the official state textbook database in school or Library and it makes connection between casher or admin or anyone and books in stock very easily and control money income and data.

5.3 Future Work

In Future most of supermarkets or libraries or any small objection will use desktop systems to make interactions between customers in an easy and fast way .

Appendix
Source Code

```
// the main class
public class textbook {


    public static void main(String[] args) {
        Home home = new Home();
        home.setVisible(true);
    }


}


// Drag Class
// imported libraries
import java.awt.Point;
import java.awt.event.*;
import javax.swing.JFrame;


// class to create function to drag the form on the screen
public class Drag extends MouseAdapter{
    private final JFrame frame;
        private Point mouseDownCompCoords = null;
        public Drag(JFrame frame) {
            this.frame = frame;
        }
```

```
        @Override
        public void mouseReleased(MouseEvent e) {
            mouseDownCompCoords = null;
        }

        @Override
        public void mousePressed(MouseEvent e) {
            mouseDownCompCoords = e.getPoint();
        }

        @Override
        public void mouseDragged(MouseEvent e) {
            Point currCoords = e.getLocationOnScreen();
            frame.setLocation(currCoords.x - mouseDownCompCoords.x, currCoords.y - mouseDownCompCoords.y);
        }
}

// Home Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;
import net.proteanit.sql.DbUtils;
```

```
public class Home extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final JTextField textfield0;
    private final Color color1, color2, color3;
    private final Font font;
    private final JTable table;
    private final JScrollPane scrollPane;
    private final JComboBox combobox;
    private Connection connection;
    private PreparedStatement preparedstatement;
    private ResultSet resultset;
    private String sql;

    // the class constructor
    public Home(){

        // form implementation
        this.setLocation(280, 80);
        this.setSize(800, 600);
        this.setUndecorated(true);
        this.setResizable(false);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
action = new Action();
panel = new JPanel();
label = new JLabel[20];
textfield0 = new JTextField();
table = new JTable();
combobox = new JComboBox();
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// function call
viewcategory();

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// comboBox implementation
combobox.setBounds(275, 272, 250, 30);
panel.add(combobox);
combobox.addMouseListener(action);

// scrollPane implementation
```

```
panel.add(table);
scrollPane = new JScrollPane(table);
scrollPane.setBounds(20, 320, 760, 250);
scrollPane.setEnabled(false);
panel.add(scrollPane);

// text field implementation
textfield0.setBounds(275, 100, 300, 35);
textfield0.setFont(font);
panel.add(textfield0);

// labels implementation
label[0] = new JLabel("  Home");
label[0].setBackground(color3);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(0, 0, 700, 35);
label[0].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[0]);

label[1] = new JLabel("   X");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(749, 1, 50, 34);
label[1].setFont(font);
```

```
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("  ---");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(699, 1, 50, 34);
label[2].setFont(font);
panel.add(label[2]);
label[2].addMouseListener(action);

label[3] = new JLabel("        LogIn");
label[3].setBackground(color3);
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(605, 50, 150, 35);
label[3].setFont(font);
panel.add(label[3]);
label[3].addMouseListener(action);

label[4] = new JLabel("  Search By Category:");
label[4].setBackground(color2);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(20, 270, 250, 35);
```

```
label[4].setFont(font);
panel.add(label[4]);

label[5] = new JLabel("  Search By Book Name:");
label[5].setBackground(color2);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(20, 100, 250, 35);
label[5].setFont(font);
panel.add(label[5]);

label[6] = new JLabel("         Search");
label[6].setBackground(color3);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
label[6].setBounds(590, 100, 180, 35);
label[6].setFont(font);
panel.add(label[6]);
label[6].addMouseListener(action);

label[7] = new JLabel("Book Code");
label[7].setBackground(color2);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
label[7].setBounds(30, 170, 150, 30);
label[7].setFont(font);
```

```
panel.add(label[7]);


label[8] = new JLabel("Book Name");
label[8].setBackground(color2);
label[8].setOpaque(true);
label[8].setForeground(Color.WHITE);
label[8].setBounds(200, 170, 150, 30);
label[8].setFont(font);
panel.add(label[8]);


label[9] = new JLabel("Book Price");
label[9].setBackground(color2);
label[9].setOpaque(true);
label[9].setForeground(Color.WHITE);
label[9].setBounds(400, 170, 150, 30);
label[9].setFont(font);
panel.add(label[9]);


label[10] = new JLabel("Status");
label[10].setBackground(color2);
label[10].setOpaque(true);
label[10].setForeground(Color.WHITE);
label[10].setBounds(560, 170, 150, 30);
label[10].setFont(font);
panel.add(label[10]);
```

```
label[11] = new JLabel("");
label[11].setBackground(color2);
label[11].setOpaque(true);
label[11].setForeground(Color.WHITE);
label[11].setBounds(30, 220, 150, 30);
label[11].setFont(font);
panel.add(label[11]);

label[12] = new JLabel("");
label[12].setBackground(color2);
label[12].setOpaque(true);
label[12].setForeground(Color.WHITE);
label[12].setBounds(200, 220, 300, 30);
label[12].setFont(font);
panel.add(label[12]);

label[13] = new JLabel("");
label[13].setBackground(color2);
label[13].setOpaque(true);
label[13].setForeground(Color.WHITE);
label[13].setBounds(400, 220, 150, 30);
label[13].setFont(font);
panel.add(label[13]);

label[14] = new JLabel("");
label[14].setBackground(color2);
```

```
    label[14].setOpaque(true);
    label[14].setForeground(Color.WHITE);
    label[14].setBounds(560, 220, 150, 30);
    label[14].setFont(font);
    panel.add(label[14]);

    label[15] = new JLabel("          Apply");
    label[15].setBackground(color3);
    label[15].setOpaque(true);
    label[15].setForeground(Color.WHITE);
    label[15].setBounds(590, 270, 180, 35);
    label[15].setFont(font);
    panel.add(label[15]);
    label[15].addMouseListener(action);

    Drag frameDragListener = new Drag(this);
    label[0].addMouseListener(frameDragListener);
    label[0].addMouseMotionListener(frameDragListener);
}

// login function to go to login form
private void login(){
    this.setVisible(false);
    Login login = new Login();
    login.setVisible(true);
}
```

```
    // viewcategory function to view all categories in the book table into comboBox
    private void viewcategory(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT DISTINCT category FROM books";
            preparedstatement = connection.prepareStatement(sql);
            resultset = preparedstatement.executeQuery();
            while(resultset.next()){
                String s = resultset.getString("category");
                combobox.addItem(s.trim());
            }
            connection.close();
        }catch(SQLException ex){
            JOptionPane.showMessageDialog(null, "Cannot View!"+ex.getMessage());
        }
    }

    // view function to view all books with the same category
    private void view(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT code,name,price,InStore AS Available FROM books WHERE
category = ? AND InStore > 0";
            preparedstatement = connection.prepareStatement(sql);
```

```
            preparedstatement.setString(1,combobox.getSelectedItem().toString());
            resultset = preparedstatement.executeQuery();
            table.setModel(DbUtils.resultSetToTableModel(resultset));
            connection.close();
        }catch(SQLException ex){
            JOptionPane.showMessageDialog(null, "Cannot View!"+ex.getMessage());
        }
    }

    // available function to check if a book is available or not in the store
    private void available(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT InStore FROM books WHERE name = ? AND InStore> 0";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            resultset = preparedstatement.executeQuery();
            if (resultset.next()) {
                label[14].setText("Available");
            }else{
                label[14].setText("Not Available");
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
```

```
    }
}

// search function to search for specific book data in the database
private void search(){
    try{
        connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
        sql = "SELECT code,name,price FROM books WHERE name = ?";
        preparedstatement = connection.prepareStatement(sql);
        preparedstatement.setString(1,textfield0.getText());
        resultset = preparedstatement.executeQuery();
        if(resultset.next()){
            label[11].setText(resultset.getString(1));
            label[12].setText(resultset.getString(2));
            label[13].setText(resultset.getString(3));
            available();
        }else{
            available();
        }
        connection.close();
    }catch(SQLException e){
        JOptionPane.showMessageDialog(null, e);
    }
}
```

```
// function to minimize the form
private void minimize(){
    this.setState(JFrame.ICONIFIED);
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[1]) {
            System.exit(0);
        }
        if (me.getSource() == label[2]) {
            minimize();
        }
        if (me.getSource() == label[3]) {
            login();
        }
        if (me.getSource() == label[6]) {
            search();
        }
        if (me.getSource() == label[15]) {
            view();
        }
    }
```

```
@Override
public void mousePressed(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.brighter());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1.brighter());
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1.brighter());
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1.brighter());
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[2]) {
```

```
            label[2].setBackground(color1);
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color1);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
        if (me.getSource() == label[15]) {
            label[15].setBackground(color1);
        }
    }

    @Override
    public void mouseEntered(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1);
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color1);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
```

```
            }
            if (me.getSource() == label[15]) {
                label[15].setBackground(color1);
            }
        }

        @Override
        public void mouseExited(MouseEvent me) {
            if (me.getSource() == label[1]) {
                label[1].setBackground(color3);
            }
            if (me.getSource() == label[2]) {
                label[2].setBackground(color3);
            }
            if (me.getSource() == label[3]) {
                label[3].setBackground(color3);
            }
            if (me.getSource() == label[6]) {
                label[6].setBackground(color3);
            }
            if (me.getSource() == label[15]) {
                label[15].setBackground(color3);
            }
        }
    }
}
```

```
// Login Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class Login extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final JTextField textfield0;
    private final JPasswordField passwordfield;
    private final JRadioButton radiobutton0, radiobutton1;
    private final ButtonGroup buttongroup;
    private final Color color1, color2, color3;
    private final Font font;
    private Connection connection;
    private PreparedStatement preparedstatement;
    private ResultSet resultset;
    private String sql;

    // the class constructor
    public Login(){
```

```
// form implementation
this.setLocation(500, 100);
this.setSize(450, 450);
this.setUndecorated(true);
this.setResizable(false);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

action = new Action();
panel = new JPanel();
label = new JLabel[10];
textfield0 = new JTextField();
passwordfield = new JPasswordField();
buttongroup = new ButtonGroup();
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// radio buttons implementation
```

```
radiobutton0 = new JRadioButton("Admin");
radiobutton0.setBackground(color2);
radiobutton0.setBounds(110, 300, 100, 30);
radiobutton0.setForeground(Color.WHITE);
radiobutton0.setFont(font);
buttongroup.add(radiobutton0);
panel.add(radiobutton0);

radiobutton1 = new JRadioButton("Seller");
radiobutton1.setBackground(color2);
radiobutton1.setBounds(220, 300, 100, 30);
radiobutton1.setForeground(Color.WHITE);
radiobutton1.setFont(font);
buttongroup.add(radiobutton1);
panel.add(radiobutton1);

// text field implementation
textfield0.setBounds(100, 140, 250, 30);
textfield0.setFont(font);
panel.add(textfield0);

passwordfield.setBounds(100, 220, 250, 30);
passwordfield.setFont(font);
panel.add(passwordfield);

// labels implementation
```

```
label[0] = new JLabel("  Login");
label[0].setBackground(color3);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(0, 0, 350, 35);
label[0].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[0]);

label[1] = new JLabel("   X");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(400, 0, 50, 35);
label[1].setFont(font);
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("  ---");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(350, 0, 50, 35);
label[2].setFont(font);
panel.add(label[2]);
label[2].addMouseListener(action);
```

```
label[3] = new JLabel("User Name");
label[3].setBackground(color2);
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(100, 100, 150, 30);
label[3].setFont(font);
panel.add(label[3]);

label[4] = new JLabel("Password");
label[4].setBackground(color2);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(100, 180, 150, 30);
label[4].setFont(font);
panel.add(label[4]);

label[5] = new JLabel("Type");
label[5].setBackground(color2);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(100, 260, 150, 30);
label[5].setFont(font);
panel.add(label[5]);

label[6] = new JLabel("        Login");
label[6].setBackground(color3);
```

```
    label[6].setOpaque(true);
    label[6].setForeground(Color.WHITE);
    label[6].setBounds(150, 350, 160, 30);
    label[6].setFont(font);
    panel.add(label[6]);
    label[6].addMouseListener(action);

    label[7] = new JLabel("         Back");
    label[7].setBackground(color3);
    label[7].setOpaque(true);
    label[7].setForeground(Color.WHITE);
    label[7].setBounds(150, 390, 160, 30);
    label[7].setFont(font);
    panel.add(label[7]);
    label[7].addMouseListener(action);

    Drag frameDragListener = new Drag(this);
    label[0].addMouseListener(frameDragListener);
    label[0].addMouseMotionListener(frameDragListener);
}

// back function to retun to Home Page
private void back(){
    this.setVisible(false);
    Home home = new Home();
    home.setVisible(true);
```

```
    }

    // to admin function to go to Admin form after login
    private void toadmin(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM admin WHERE username = ? and password = ?";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield0.getText());
            preparedstatement.setString(2,passwordfield.getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
                String user = textfield0.getText();
                Admin admin = new Admin();
                admin.setVisible(true);
                this.setVisible(true);
                this.setVisible(false);
                admin.username(user);
            }else{
                JOptionPane.showMessageDialog(null, "NO User Found!");
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
```

```
    }

    // to seller function to go to Seller form after login
    private void toseller(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM seller WHERE username = ? and password = ?";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield0.getText());
            preparedstatement.setString(2,passwordfield.getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
                String user = textfield0.getText();
                Seller seller = new Seller();
                seller.setVisible(true);
                this.setVisible(true);
                this.setVisible(false);
                seller.username(user);
            }else{
                JOptionPane.showMessageDialog(null, "NO User Found!");
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
```

```
}

// function to minimize the form
private void minimize(){
    this.setState(JFrame.ICONIFIED);
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[1]) {
            System.exit(0);
        }
        if (me.getSource() == label[2]) {
            minimize();
        }
        if (me.getSource() == label[6]) {
            if (radiobutton0.isSelected()) {
                toadmin();
            }else if (radiobutton1.isSelected()) {
                toseller();
            }
        }
        if (me.getSource() == label[7]) {
```

```
            back();
        }
    }

    @Override
    public void mousePressed(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(Color.RED.brighter());
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1.brighter());
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1.brighter());
        }
        if (me.getSource() == label[7]) {
            label[7].setBackground(color1.brighter());
        }
    }

    @Override
    public void mouseReleased(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[2]) {
```

```
            label[2].setBackground(color1);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
        if (me.getSource() == label[7]) {
            label[7].setBackground(color1);
        }
    }

    @Override
    public void mouseEntered(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
        if (me.getSource() == label[7]) {
            label[7].setBackground(color1);
        }
    }
```

```
        @Override
        public void mouseExited(MouseEvent me) {
            if (me.getSource() == label[1]) {
                label[1].setBackground(color3);
            }
            if (me.getSource() == label[2]) {
                label[2].setBackground(color3);
            }
            if (me.getSource() == label[6]) {
                label[6].setBackground(color3);
            }
            if (me.getSource() == label[7]) {
                label[7].setBackground(color3);
            }
        }
    }
}

// Admin Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Admin extends JFrame{
```

```
private final Action action;
private final JPanel panel;
private final JLabel[] label;
private final Color color1, color2, color3;
private final Font font;

// the class constructor
public Admin(){

    // form implementation
    this.setLocation(500, 120);
    this.setSize(350, 400);
    this.setUndecorated(true);
    this.setResizable(false);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    action = new Action();
    panel = new JPanel();
    label = new JLabel[10];
    color1 = new Color(60, 60, 60);
    color2 = new Color(30, 30, 30);
    color3 = new Color(15, 15, 15);
    font = new Font("seirf", Font.BOLD, 22);

    // background implementation
    panel.setBackground(color2);
```

```
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// labels implementation
label[0] = new JLabel("  Admin");
label[0].setBackground(color3);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(0, 0, 120, 35);
label[0].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[0]);

label[1] = new JLabel("   X");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(300, 0, 50, 35);
label[1].setFont(font);
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("  ---");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
```

```
label[2].setBounds(250, 0, 50, 35);
label[2].setFont(font);
panel.add(label[2]);
label[2].addMouseListener(action);

label[3] = new JLabel("        Employees");
label[3].setBackground(color3);
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(80, 100, 200, 30);
label[3].setFont(font);
panel.add(label[3]);
label[3].addMouseListener(action);

label[4] = new JLabel("          Books");
label[4].setBackground(color3);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(80, 150, 200, 30);
label[4].setFont(font);
panel.add(label[4]);
label[4].addMouseListener(action);

label[5] = new JLabel("          Stock");
label[5].setBackground(color3);
label[5].setOpaque(true);
```

```
label[5].setForeground(Color.WHITE);
label[5].setBounds(80, 200, 200, 30);
label[5].setFont(font);
panel.add(label[5]);
label[5].addMouseListener(action);

label[6] = new JLabel("          Profile");
label[6].setBackground(color3);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
label[6].setBounds(80, 250, 200, 30);
label[6].setFont(font);
panel.add(label[6]);
label[6].addMouseListener(action);

label[7] = new JLabel("          Logout");
label[7].setBackground(color3);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
label[7].setBounds(80, 300, 200, 30);
label[7].setFont(font);
panel.add(label[7]);
label[7].addMouseListener(action);

label[8] = new JLabel("");
label[8].setBackground(color3);
```

```
    label[8].setOpaque(true);
    label[8].setForeground(Color.WHITE);
    label[8].setBounds(120, 0, 130, 35);
    label[8].setFont(new Font("seirf", Font.BOLD, 25));
    panel.add(label[8]);

    Drag frameDragListener = new Drag(this);
    label[0].addMouseListener(frameDragListener);
    label[0].addMouseMotionListener(frameDragListener);
    label[8].addMouseListener(frameDragListener);
    label[8].addMouseMotionListener(frameDragListener);
}

// function to minimize the form
private void minimize(){
    this.setState(JFrame.ICONIFIED);
}

// function to go to employee form
private void toemployee(){
    this.setVisible(false);
    Employee employee = new Employee();
    employee.setVisible(true);
    employee.username(label[8].getText());
}
```

```
// function to go to books form
private void tobooks(){
    this.setVisible(false);
    Books book = new Books();
    book.setVisible(true);
    book.username(label[8].getText());
}

// function to go to stock form
private void tostock(){
    this.setVisible(false);
    Stock stock = new Stock();
    stock.setVisible(true);
    stock.username(label[8].getText());
}

// function to go to profile form
private void toprofile(){
    this.setVisible(false);
    Profile profile = new Profile();
    profile.setVisible(true);
    profile.username(label[8].getText(), "admin");
    profile.useradmin();
}

// username function to get the username from login
```

```
public void username(String user){
    label[8].setText(user);
}

// function to go to home form
private void tohome(){
    this.setVisible(false);
    Home home = new Home();
    home.setVisible(true);
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[1]) {
            System.exit(0);
        }
        if (me.getSource() == label[2]) {
            minimize();
        }
        if (me.getSource() == label[3]) {
            toemployee();
        }
        if (me.getSource() == label[4]) {
```

```
            tobooks();
        }
        if (me.getSource() == label[5]) {
            tostock();
        }
        if (me.getSource() == label[6]) {
            toprofile();
        }
        if (me.getSource() == label[7]) {
            tohome();
        }
    }

    @Override
    public void mousePressed(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(Color.RED.brighter());
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1.brighter());
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color1.brighter());
        }
        if (me.getSource() == label[4]) {
            label[4].setBackground(color1.brighter());
```

```
    }
    if (me.getSource() == label[5]) {
        label[5].setBackground(color1.brighter());
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1.brighter());
    }
    if (me.getSource() == label[7]) {
        label[7].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1);
    }
```

```
    if (me.getSource() == label[5]) {
        label[5].setBackground(color1);
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1);
    }
    if (me.getSource() == label[7]) {
        label[7].setBackground(color1);
    }
}

@Override
public void mouseEntered(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1);
    }
    if (me.getSource() == label[5]) {
```

```
            label[5].setBackground(color1);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
        if (me.getSource() == label[7]) {
            label[7].setBackground(color1);
        }
    }

    @Override
    public void mouseExited(MouseEvent me) {
        if (me.getSource() == label[1]) {
            label[1].setBackground(color3);
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color3);
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color3);
        }
        if (me.getSource() == label[4]) {
            label[4].setBackground(color3);
        }
        if (me.getSource() == label[5]) {
            label[5].setBackground(color3);
```

```
            }
            if (me.getSource() == label[6]) {
                label[6].setBackground(color3);
            }
            if (me.getSource() == label[7]) {
                label[7].setBackground(color3);
            }
        }
    }
}

// Employee Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class Employee extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final JTextField textfield0, textfield1, textfield2,
            textfield3, textfield4, textfield5, textfield6;
    private final JRadioButton radiobutton0, radiobutton1, radiobutton2, radiobutton3;
```

```
private final ButtonGroup buttongroup1, buttongroup2;
private final Color color1, color2, color3;
private final Font font;
private Connection connection;
private PreparedStatement preparedstatement;
private ResultSet resultset;
private String sql;

// the class constructor
public Employee(){

    // form implementation
    this.setLocation(310, 90);
    this.setSize(800, 510);
    this.setUndecorated(true);
    this.setResizable(false);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    action = new Action();
    panel = new JPanel();
    label = new JLabel[20];
    textfield0 = new JTextField();
    textfield1 = new JTextField();
    textfield2 = new JTextField();
    textfield3 = new JTextField();
    textfield4 = new JTextField();
```

```
textfield5 = new JTextField();
textfield6 = new JTextField();
buttongroup1 = new ButtonGroup();
buttongroup2 = new ButtonGroup();
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// radio buttons implementation
radiobutton0 = new JRadioButton("ID");
radiobutton0.setBackground(color2);
radiobutton0.setBounds(575, 170, 50, 30);
radiobutton0.setForeground(Color.WHITE);
radiobutton0.setFont(new Font("seirf", Font.BOLD, 18));
buttongroup1.add(radiobutton0);
panel.add(radiobutton0);

radiobutton1 = new JRadioButton("User Name");
radiobutton1.setBackground(color2);
```

```
radiobutton1.setBounds(630, 170, 130, 30);
radiobutton1.setForeground(Color.WHITE);
radiobutton1.setFont(new Font("seirf", Font.BOLD, 18));
buttongroup1.add(radiobutton1);
panel.add(radiobutton1);

radiobutton2 = new JRadioButton("Admin");
radiobutton2.setBackground(color2);
radiobutton2.setBounds(560, 250, 90, 30);
radiobutton2.setForeground(Color.WHITE);
radiobutton2.setFont(font);
buttongroup2.add(radiobutton2);
panel.add(radiobutton2);

radiobutton3 = new JRadioButton("Seller");
radiobutton3.setBackground(color2);
radiobutton3.setBounds(650, 250, 90, 30);
radiobutton3.setForeground(Color.WHITE);
radiobutton3.setFont(font);
buttongroup2.add(radiobutton3);
panel.add(radiobutton3);

// text field implementation
textfield0.setBounds(200, 150, 325, 30);
textfield0.setFont(font);
panel.add(textfield0);
```

```
textfield1.setBounds(200, 200, 325, 30);
textfield1.setFont(font);
panel.add(textfield1);

textfield2.setBounds(200, 250, 325, 30);
textfield2.setFont(font);
panel.add(textfield2);

textfield3.setBounds(200, 300, 325, 30);
textfield3.setFont(font);
panel.add(textfield3);

textfield4.setBounds(200, 350, 325, 30);
textfield4.setFont(font);
panel.add(textfield4);

textfield5.setBounds(200, 400, 325, 30);
textfield5.setFont(font);
panel.add(textfield5);

textfield6.setBounds(30, 70, 500, 30);
textfield6.setFont(font);
panel.add(textfield6);

// labels implementation
```

```
label[0] = new JLabel("  ID");
label[0].setBackground(color2);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(30, 150, 150, 30);
label[0].setFont(font);
panel.add(label[0]);

label[1] = new JLabel("  Name");
label[1].setBackground(color2);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(30, 200, 150, 30);
label[1].setFont(font);
panel.add(label[1]);

label[2] = new JLabel("  User Name");
label[2].setBackground(color2);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(30, 250, 150, 30);
label[2].setFont(font);
panel.add(label[2]);

label[3] = new JLabel("  Password");
label[3].setBackground(color2);
```

```
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(30, 300, 150, 30);
label[3].setFont(font);
panel.add(label[3]);

label[4] = new JLabel("  Phone");
label[4].setBackground(color2);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(30, 350, 150, 30);
label[4].setFont(font);
panel.add(label[4]);

label[5] = new JLabel("Type");
label[5].setBackground(color2);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(560, 210, 150, 30);
label[5].setFont(font);
panel.add(label[5]);

label[6] = new JLabel("  Salary");
label[6].setBackground(color2);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
```

```
label[6].setBounds(30, 400, 150, 30);
label[6].setFont(font);
panel.add(label[6]);

label[9] = new JLabel("          Search");
label[9].setBackground(color3);
label[9].setOpaque(true);
label[9].setForeground(Color.WHITE);
label[9].setBounds(560, 70, 200, 30);
label[9].setFont(font);
panel.add(label[9]);
label[9].addMouseListener(action);

label[10] = new JLabel("       New Search");
label[10].setBackground(color3);
label[10].setOpaque(true);
label[10].setForeground(Color.WHITE);
label[10].setBounds(560, 120, 200, 30);
label[10].setFont(font);
panel.add(label[10]);
label[10].addMouseListener(action);

label[11] = new JLabel("          Back");
label[11].setBackground(color3);
label[11].setOpaque(true);
label[11].setForeground(Color.WHITE);
```

```
label[11].setBounds(560, 450, 200, 30);
label[11].setFont(font);
panel.add(label[11]);
label[11].addMouseListener(action);

label[12] = new JLabel("     Add Employee");
label[12].setBackground(color3);
label[12].setOpaque(true);
label[12].setForeground(Color.WHITE);
label[12].setBounds(560, 300, 200, 30);
label[12].setFont(font);
panel.add(label[12]);
label[12].addMouseListener(action);

label[13] = new JLabel("     Edit Employee");
label[13].setBackground(color3);
label[13].setOpaque(true);
label[13].setForeground(Color.WHITE);
label[13].setBounds(560, 350, 200, 30);
label[13].setFont(font);
panel.add(label[13]);
label[13].addMouseListener(action);

label[14] = new JLabel("   Delete Employee");
label[14].setBackground(color3);
label[14].setOpaque(true);
```

```
label[14].setForeground(Color.WHITE);
label[14].setBounds(560, 400, 200, 30);
label[14].setFont(font);
panel.add(label[14]);
label[14].addMouseListener(action);

label[15] = new JLabel("   X");
label[15].setBackground(color3);
label[15].setOpaque(true);
label[15].setForeground(Color.WHITE);
label[15].setBounds(750, 0, 50, 35);
label[15].setFont(font);
panel.add(label[15]);
label[15].addMouseListener(action);

label[16] = new JLabel("  ---");
label[16].setBackground(color3);
label[16].setOpaque(true);
label[16].setForeground(Color.WHITE);
label[16].setBounds(700, 0, 50, 35);
label[16].setFont(font);
panel.add(label[16]);
label[16].addMouseListener(action);

label[17] = new JLabel("  Employee");
label[17].setBackground(color3);
```

```
    label[17].setOpaque(true);
    label[17].setForeground(Color.WHITE);
    label[17].setBounds(0, 0, 180, 35);
    label[17].setFont(new Font("seirf", Font.BOLD, 30));
    panel.add(label[17]);

    label[18] = new JLabel("");
    label[18].setBackground(color3);
    label[18].setOpaque(true);
    label[18].setForeground(Color.WHITE);
    label[18].setBounds(180, 0, 520, 35);
    label[18].setFont(new Font("seirf", Font.BOLD, 25));
    panel.add(label[18]);

    Drag frameDragListener = new Drag(this);
    label[17].addMouseListener(frameDragListener);
    label[17].addMouseMotionListener(frameDragListener);
    label[18].addMouseListener(frameDragListener);
    label[18].addMouseMotionListener(frameDragListener);
}

// function to minimize the form
private void minimize(){
    this.setState(JFrame.ICONIFIED);
}
```

```
// back function to return to admin form
private void back(){
    Admin admin = new Admin();
    admin.setVisible(true);
    this.setVisible(true);
    this.setVisible(false);
    admin.username(label[18].getText());
}

// username function to get the username from login
public void username(String user){
    label[18].setText(user);
}

// clear function to clear all text field after each process
private void clear(){
    textfield0.setText("");
    textfield1.setText("");
    textfield2.setText("");
    textfield3.setText("");
    textfield4.setText("");
    textfield5.setText("");
    textfield6.setText("");
}

// insert function to insert employees data into the database
```

```
    private void insert(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton2.isSelected()) {
                sql = "INSERT INTO admin(id, name, username, password, phone, salary)
VALUES(?,?,?,?,?,?)";
            }
            else if(radiobutton3.isSelected()){
                sql = "INSERT INTO seller(id, name, username, password, phone, salary)
VALUES(?,?,?,?,?,?)";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            preparedstatement.setString(2, textfield1.getText());
            preparedstatement.setString(3, textfield2.getText());
            preparedstatement.setString(4, textfield3.getText());
            preparedstatement.setString(5, textfield4.getText());
            preparedstatement.setString(6, textfield5.getText());
            if(preparedstatement.executeUpdate()>0){
            JOptionPane.showMessageDialog(null, "Employee Data Added Successfully!");
            clear();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
```

```
    }

    // search function to search for specific employee data in the database
    private void search(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                if (radiobutton2.isSelected()) {
                    sql = "SELECT * FROM admin WHERE id = ?";
                }
                else if(radiobutton3.isSelected()){
                    sql = "SELECT * FROM seller WHERE id = ?";
                }
            }
            else if(radiobutton1.isSelected()){
                if (radiobutton2.isSelected()) {
                    sql = "SELECT * FROM admin WHERE username = ?";
                }
                else if(radiobutton3.isSelected()){
                    sql = "SELECT * FROM seller WHERE username = ?";
                }
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield6.getText());
            resultset = preparedstatement.executeQuery();
```

```
            if(resultset.next()){
                textfield0.setText(resultset.getString(1));
                textfield1.setText(resultset.getString(2));
                textfield2.setText(resultset.getString(3));
                textfield3.setText(resultset.getString(4));
                textfield4.setText(resultset.getString(5));
                textfield5.setText(resultset.getString(7));
            }else{
                JOptionPane.showMessageDialog(null, "NO Employee Found!");
                clear();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, "Please Select Search Method");
        }
    }

    // update function to update specific employee data in the database
    private void update(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                if (radiobutton2.isSelected()) {
                    sql = "UPDATE admin SET id = ?, name = ?, username = ?, password = ?, "
                    + "phone = ?, salary = ? WHERE id = ?";
```

```
    }
    else if(radiobutton3.isSelected()){
        sql = "UPDATE seller SET id = ?, name = ?, username = ?, password = ?, "
        + "phone = ?, salary = ? WHERE id = ?";
    }
}
else if(radiobutton1.isSelected()){
    if (radiobutton2.isSelected()) {
        sql = "UPDATE admin SET id = ?, name = ?, username = ?, password = ?, "
        + "phone = ?, salary = ? WHERE name = ?";
    }
    else if(radiobutton3.isSelected()){
        sql = "UPDATE seller SET id = ?, name = ?, username = ?, password = ?, "
        + "phone = ?, salary = ? WHERE name = ?";
    }
}
preparedstatement = connection.prepareStatement(sql);
preparedstatement.setString(1, textfield0.getText());
preparedstatement.setString(2, textfield1.getText());
preparedstatement.setString(3, textfield2.getText());
preparedstatement.setString(4, textfield3.getText());
preparedstatement.setString(5, textfield4.getText());
preparedstatement.setString(6, textfield5.getText());
preparedstatement.setString(7, textfield6.getText());
if (preparedstatement.executeUpdate()>0) {
```

```
            JOptionPane.showMessageDialog(null, "Employee Data Updated
Successfully!");
            clear();
        }
        connection.close();
    }catch(SQLException e){
        JOptionPane.showMessageDialog(null, e);
    }
}

// delete function to delete specific book data from the database
private void delete(){
    try{
        connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
        if (radiobutton0.isSelected()) {
            if (radiobutton2.isSelected()) {
                sql = "DELETE FROM admin WHERE id = ?";
            }
            else if(radiobutton3.isSelected()){
                sql = "DELETE FROM seller WHERE id = ?";
            }
        }
        else if(radiobutton1.isSelected()){
            if (radiobutton2.isSelected()) {
                sql = "DELETE FROM admin WHERE name = ?";
            }
```

```
            else if(radiobutton3.isSelected()){
                sql = "DELETE FROM seller WHERE name = ?";
            }
        }
        preparedstatement = connection.prepareStatement(sql);
        preparedstatement.setString(1,textfield6.getText());
        if(preparedstatement.executeUpdate()>0){
            JOptionPane.showMessageDialog(null, "Employee Data Deleted Successfully!");
            clear();
        }
        connection.close();
    }catch(SQLException e){
        JOptionPane.showMessageDialog(null, e);
    }
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[9]) {
            search();
        }
        if (me.getSource() == label[10]) {
            clear();
```

```
    }
    if (me.getSource() == label[11]) {
        back();
    }
    if (me.getSource() == label[12]) {
        insert();
    }
    if (me.getSource() == label[13]) {
        update();
    }
    if (me.getSource() == label[14]) {
        delete();
    }
    if (me.getSource() == label[15]) {
        System.exit(0);
    }
    if (me.getSource() == label[16]) {
        minimize();
    }
}

@Override
public void mousePressed(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color1.brighter());
    }
```

```
    if (me.getSource() == label[10]) {
        label[10].setBackground(color1.brighter());
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color1.brighter());
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color1.brighter());
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color1.brighter());
    }
    if (me.getSource() == label[14]) {
        label[14].setBackground(color1.brighter());
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(Color.RED.brighter());
    }
    if (me.getSource() == label[16]) {
        label[16].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[9]) {
```

```
            label[9].setBackground(color1);
        }
        if (me.getSource() == label[10]) {
            label[10].setBackground(color1);
        }
        if (me.getSource() == label[11]) {
            label[11].setBackground(color1);
        }
        if (me.getSource() == label[12]) {
            label[12].setBackground(color1);
        }
        if (me.getSource() == label[13]) {
            label[13].setBackground(color1);
        }
        if (me.getSource() == label[14]) {
            label[14].setBackground(color1);
        }
        if (me.getSource() == label[15]) {
            label[15].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[16]) {
            label[16].setBackground(color1);
        }
    }

    @Override
```

```
public void mouseEntered(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color1);
    }
    if (me.getSource() == label[10]) {
        label[10].setBackground(color1);
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color1);
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color1);
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color1);
    }
    if (me.getSource() == label[14]) {
        label[14].setBackground(color1);
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[16]) {
        label[16].setBackground(color1);
    }
}
```

```
@Override
public void mouseExited(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color3);
    }
    if (me.getSource() == label[10]) {
        label[10].setBackground(color3);
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color3);
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color3);
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color3);
    }
    if (me.getSource() == label[14]) {
        label[14].setBackground(color3);
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(color3);
    }
    if (me.getSource() == label[16]) {
        label[16].setBackground(color3);
```

```
            }
        }
    }
}

// Books Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class Books extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final JTextField textfield0, textfield1, textfield2, textfield3, textfield4,
            textfield5, textfield6, textfield7, textfield8, textfield9;
    private final JRadioButton radiobutton0, radiobutton1;
    private final ButtonGroup buttongroup;
    private final Color color1, color2, color3;
    private final Font font;
    private Connection connection;
    private PreparedStatement preparedstatement;
    private ResultSet resultset;
```

```
private String sql;

// the class constructor
public Books(){

    // form implementation
    this.setLocation(310, 80);
    this.setSize(800, 600);
    this.setUndecorated(true);
    this.setResizable(false);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    action = new Action();
    panel = new JPanel();
    label = new JLabel[20];
    textfield0 = new JTextField();
    textfield1 = new JTextField();
    textfield2 = new JTextField();
    textfield3 = new JTextField();
    textfield4 = new JTextField();
    textfield5 = new JTextField();
    textfield6 = new JTextField();
    textfield7 = new JTextField();
    textfield8 = new JTextField();
    textfield9 = new JTextField();
    buttongroup = new ButtonGroup();
```

```
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// radio buttons implementation
radiobutton0 = new JRadioButton("Code");
radiobutton0.setBackground(color2);
radiobutton0.setBounds(570, 130, 90, 30);
radiobutton0.setForeground(Color.WHITE);
radiobutton0.setFont(font);
buttongroup.add(radiobutton0);
panel.add(radiobutton0);

radiobutton1 = new JRadioButton("Name");
radiobutton1.setBackground(color2);
radiobutton1.setBounds(665, 130, 90, 30);
radiobutton1.setForeground(Color.WHITE);
radiobutton1.setFont(font);
buttongroup.add(radiobutton1);
```

```
panel.add(radiobutton1);

// text field implementation
textfield0.setBounds(200, 150, 325, 30);
textfield0.setFont(font);
panel.add(textfield0);

textfield1.setBounds(200, 200, 325, 30);
textfield1.setFont(font);
panel.add(textfield1);

textfield2.setBounds(200, 250, 325, 30);
textfield2.setFont(font);
panel.add(textfield2);

textfield3.setBounds(200, 300, 325, 30);
textfield3.setFont(font);
panel.add(textfield3);

textfield4.setBounds(200, 350, 325, 30);
textfield4.setFont(font);
panel.add(textfield4);

textfield5.setBounds(200, 400, 325, 30);
textfield5.setFont(font);
panel.add(textfield5);
```

```
textfield6.setBounds(200, 450, 325, 30);
textfield6.setFont(font);
panel.add(textfield6);

textfield7.setBounds(200, 500, 325, 30);
textfield7.setFont(font);
panel.add(textfield7);

textfield8.setBounds(200, 550, 325, 30);
textfield8.setFont(font);
panel.add(textfield8);

textfield9.setBounds(30, 70, 500, 30);
textfield9.setFont(font);
panel.add(textfield9);

// labels implementation
label[0] = new JLabel("  Book Code");
label[0].setBackground(color2);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(30, 150, 150, 30);
label[0].setFont(font);
panel.add(label[0]);
```

```
label[1] = new JLabel("  Book Name");
label[1].setBackground(color2);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(30, 200, 150, 30);
label[1].setFont(font);
panel.add(label[1]);

label[2] = new JLabel("  Category");
label[2].setBackground(color2);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(30, 250, 150, 30);
label[2].setFont(font);
panel.add(label[2]);

label[3] = new JLabel("  Price");
label[3].setBackground(color2);
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(30, 300, 150, 30);
label[3].setFont(font);
panel.add(label[3]);

label[4] = new JLabel("  Author Name");
label[4].setBackground(color2);
```

```
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(30, 350, 150, 30);
label[4].setFont(font);
panel.add(label[4]);

label[5] = new JLabel("  Publisher");
label[5].setBackground(color2);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(30, 400, 150, 30);
label[5].setFont(font);
panel.add(label[5]);

label[6] = new JLabel("  Publish Date");
label[6].setBackground(color2);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
label[6].setBounds(30, 450, 150, 30);
label[6].setFont(font);
panel.add(label[6]);

label[7] = new JLabel("  In Store");
label[7].setBackground(color2);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
```

```
label[7].setBounds(30, 500, 150, 30);
label[7].setFont(font);
panel.add(label[7]);

label[8] = new JLabel("  In Stock");
label[8].setBackground(color2);
label[8].setOpaque(true);
label[8].setForeground(Color.WHITE);
label[8].setBounds(30, 550, 150, 30);
label[8].setFont(font);
panel.add(label[8]);

label[9] = new JLabel("          Search");
label[9].setBackground(color3);
label[9].setOpaque(true);
label[9].setForeground(Color.WHITE);
label[9].setBounds(560, 70, 200, 30);
label[9].setFont(font);
panel.add(label[9]);
label[9].addMouseListener(action);

label[10] = new JLabel("      New Search");
label[10].setBackground(color3);
label[10].setOpaque(true);
label[10].setForeground(Color.WHITE);
label[10].setBounds(560, 350, 200, 30);
```

```
label[10].setFont(font);
panel.add(label[10]);
label[10].addMouseListener(action);

label[11] = new JLabel("          Back");
label[11].setBackground(color3);
label[11].setOpaque(true);
label[11].setForeground(Color.WHITE);
label[11].setBounds(560, 550, 200, 30);
label[11].setFont(font);
panel.add(label[11]);
label[11].addMouseListener(action);

label[12] = new JLabel("       Add Book");
label[12].setBackground(color3);
label[12].setOpaque(true);
label[12].setForeground(Color.WHITE);
label[12].setBounds(560, 400, 200, 30);
label[12].setFont(font);
panel.add(label[12]);
label[12].addMouseListener(action);

label[13] = new JLabel("       Edit Book");
label[13].setBackground(color3);
label[13].setOpaque(true);
label[13].setForeground(Color.WHITE);
```

```
label[13].setBounds(560, 450, 200, 30);
label[13].setFont(font);
panel.add(label[13]);
label[13].addMouseListener(action);

label[14] = new JLabel("       Delete Book");
label[14].setBackground(color3);
label[14].setOpaque(true);
label[14].setForeground(Color.WHITE);
label[14].setBounds(560, 500, 200, 30);
label[14].setFont(font);
panel.add(label[14]);
label[14].addMouseListener(action);

label[15] = new JLabel("   X");
label[15].setBackground(color3);
label[15].setOpaque(true);
label[15].setForeground(Color.WHITE);
label[15].setBounds(750, 0, 50, 35);
label[15].setFont(font);
panel.add(label[15]);
label[15].addMouseListener(action);

label[16] = new JLabel("  ---");
label[16].setBackground(color3);
label[16].setOpaque(true);
```

```
label[16].setForeground(Color.WHITE);
label[16].setBounds(700, 0, 50, 35);
label[16].setFont(font);
panel.add(label[16]);
label[16].addMouseListener(action);

label[17] = new JLabel("  Books");
label[17].setBackground(color3);
label[17].setOpaque(true);
label[17].setForeground(Color.WHITE);
label[17].setBounds(0, 0, 130, 35);
label[17].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[17]);

label[18] = new JLabel("");
label[18].setBackground(color3);
label[18].setOpaque(true);
label[18].setForeground(Color.WHITE);
label[18].setBounds(130, 0, 670, 35);
label[18].setFont(new Font("seirf", Font.BOLD, 25));
panel.add(label[18]);

Drag frameDragListener = new Drag(this);
label[17].addMouseListener(frameDragListener);
label[17].addMouseMotionListener(frameDragListener);
label[18].addMouseListener(frameDragListener);
```

```
    label[18].addMouseMotionListener(frameDragListener);
}

// function to minimize the form
private void minimize(){
    this.setState(JFrame.ICONIFIED);
}

// back function to return to admin form
private void back(){
    Admin admin = new Admin();
    admin.setVisible(true);
    this.setVisible(true);
    this.setVisible(false);
    admin.username(label[18].getText());
}

// username function to get the username from login
public void username(String user){
    label[18].setText(user);
}

// clear function to clear all text field after each process
private void clear(){
    textfield0.setText("");
    textfield1.setText("");
```

```
        textfield2.setText("");
        textfield3.setText("");
        textfield4.setText("");
        textfield5.setText("");
        textfield6.setText("");
        textfield7.setText("");
        textfield8.setText("");
        textfield9.setText("");
    }

    // insert function to insert book data into the database
    private void insert(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "INSERT INTO books VALUES(?,?,?,?,?,?,?,?,?)";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            preparedstatement.setString(2, textfield1.getText());
            preparedstatement.setString(3, textfield2.getText());
            preparedstatement.setString(4, textfield3.getText());
            preparedstatement.setString(5, textfield4.getText());
            preparedstatement.setString(6, textfield5.getText());
            preparedstatement.setString(7, textfield6.getText());
            preparedstatement.setString(8, textfield7.getText());
            preparedstatement.setString(9, textfield8.getText());
```

```
            if(preparedstatement.executeUpdate()>0){
            JOptionPane.showMessageDialog(null, "Book Data Added Successfully!");
            clear();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // search function to search for specific book data in the database
    private void search(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                sql = "SELECT * FROM books WHERE code = ?";
            }
            else if(radiobutton1.isSelected()){
                sql = "SELECT * FROM books WHERE name = ?";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield9.getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
                textfield0.setText(resultset.getString(1));
```

```
            textfield1.setText(resultset.getString(2));
            textfield2.setText(resultset.getString(3));
            textfield3.setText(resultset.getString(4));
            textfield4.setText(resultset.getString(5));
            textfield5.setText(resultset.getString(6));
            textfield6.setText(resultset.getString(7));
            textfield7.setText(resultset.getString(8));
            textfield8.setText(resultset.getString(9));
        }else{
            JOptionPane.showMessageDialog(null, "NO Book Found!");
            clear();
        }
        connection.close();
    }catch(SQLException e){
        JOptionPane.showMessageDialog(null, "Please Select Search Method");
    }
}

// update function to update specific book data in the database
private void update(){
    try{
        connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
        if (radiobutton0.isSelected()) {
        sql = "UPDATE books SET code = ?, name = ?, category = ?, price = ?, author = ?,
publisher = ?,"
                + " publishDate = ?, InStore = ?, InStock = ? WHERE code = ?";
```

```
            }
            else if(radiobutton1.isSelected()){
            sql = "UPDATE books SET code = ?, name = ?, category = ?, price = ?, author = ?,
publisher = ?,"
                  + " publishDate = ?, InStore = ?, InStock = ? WHERE name = ?";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            preparedstatement.setString(2, textfield1.getText());
            preparedstatement.setString(3, textfield2.getText());
            preparedstatement.setString(4, textfield3.getText());
            preparedstatement.setString(5, textfield4.getText());
            preparedstatement.setString(6, textfield5.getText());
            preparedstatement.setString(7, textfield6.getText());
            preparedstatement.setString(8, textfield7.getText());
            preparedstatement.setString(9, textfield8.getText());
            preparedstatement.setString(10, textfield9.getText());
            if (preparedstatement.executeUpdate()>0) {
               JOptionPane.showMessageDialog(null, "Book Data Updated Successfully!");
               clear();
            }
            connection.close();
         }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
         }
      }
```

```
    // delete function to delete specific book data from the database
    private void delete(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
            sql = "DELETE FROM books WHERE code = ?";
            }
            else if(radiobutton1.isSelected()){
            sql = "DELETE FROM books WHERE name = ?";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield9.getText());
            if(preparedstatement.executeUpdate()>0){
                JOptionPane.showMessageDialog(null, "Book Data Deleted Successfully!");
                clear();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // private claas to contain mouse actions
    private class Action implements MouseListener{
```

```
@Override
public void mouseClicked(MouseEvent me) {
    if (me.getSource() == label[9]) {
        search();
    }
    if (me.getSource() == label[10]) {
        clear();
    }
    if (me.getSource() == label[11]) {
        back();
    }
    if (me.getSource() == label[12]) {
        insert();
    }
    if (me.getSource() == label[13]) {
        update();
    }
    if (me.getSource() == label[14]) {
        delete();
    }
    if (me.getSource() == label[15]) {
        System.exit(0);
    }
    if (me.getSource() == label[16]) {
        minimize();
```

```
    }
}

@Override
public void mousePressed(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color1.brighter());
    }
    if (me.getSource() == label[10]) {
        label[10].setBackground(color1.brighter());
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color1.brighter());
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color1.brighter());
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color1.brighter());
    }
    if (me.getSource() == label[14]) {
        label[14].setBackground(color1.brighter());
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(Color.RED.brighter());
    }
```

```
    if (me.getSource() == label[16]) {
        label[16].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color1);
    }
    if (me.getSource() == label[10]) {
        label[10].setBackground(color1);
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color1);
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color1);
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color1);
    }
    if (me.getSource() == label[14]) {
        label[14].setBackground(color1);
    }
    if (me.getSource() == label[15]) {
```

```
            label[15].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[16]) {
            label[16].setBackground(color1);
        }
    }

    @Override
    public void mouseEntered(MouseEvent me) {
        if (me.getSource() == label[9]) {
            label[9].setBackground(color1);
        }
        if (me.getSource() == label[10]) {
            label[10].setBackground(color1);
        }
        if (me.getSource() == label[11]) {
            label[11].setBackground(color1);
        }
        if (me.getSource() == label[12]) {
            label[12].setBackground(color1);
        }
        if (me.getSource() == label[13]) {
            label[13].setBackground(color1);
        }
        if (me.getSource() == label[14]) {
            label[14].setBackground(color1);
```

```
    }
    if (me.getSource() == label[15]) {
        label[15].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[16]) {
        label[16].setBackground(color1);
    }
}

@Override
public void mouseExited(MouseEvent me) {
    if (me.getSource() == label[9]) {
        label[9].setBackground(color3);
    }
    if (me.getSource() == label[10]) {
        label[10].setBackground(color3);
    }
    if (me.getSource() == label[11]) {
        label[11].setBackground(color3);
    }
    if (me.getSource() == label[12]) {
        label[12].setBackground(color3);
    }
    if (me.getSource() == label[13]) {
        label[13].setBackground(color3);
    }
```

```
            if (me.getSource() == label[14]) {
                label[14].setBackground(color3);
            }
            if (me.getSource() == label[15]) {
                label[15].setBackground(color3);
            }
            if (me.getSource() == label[16]) {
                label[16].setBackground(color3);
            }
        }
    }
}

// Stock Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.awt.print.PrinterException;
import java.sql.*;
import java.text.MessageFormat;
import java.util.logging.*;
import javax.swing.*;
import net.proteanit.sql.DbUtils;

public class Stock extends JFrame{
```

```
private final Action action;
private final JPanel panel;
private final JLabel[] label;
private final JTextField textfield0, textfield1;
private final JRadioButton radiobutton0, radiobutton1;
private final ButtonGroup buttongroup;
private final JTable table;
private final JScrollPane scrollPane;
private final Color color1, color2, color3;
private final Font font;
private Connection connection;
private PreparedStatement preparedstatement;
private ResultSet resultset;
private String sql;

// the class constructor
public Stock(){

    // form implementation
    this.setLocation(160, 60);
    this.setUndecorated(true);
    this.setSize(1080, 650);
    this.setResizable(false);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    action = new Action();
```

```
panel = new JPanel();
label = new JLabel[10];
textfield0 = new JTextField();
textfield1 = new JTextField();
buttongroup = new ButtonGroup();
table = new JTable();
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// call function
view();

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// scrollPane implementation
panel.add(table);
scrollPane = new JScrollPane(table);
scrollPane.setBounds(40,140,800,460);
scrollPane.setEnabled(false);
panel.add(scrollPane);
```

```
// radio buttons implementation
radiobutton0 = new JRadioButton("Code");
radiobutton0.setBackground(color2);
radiobutton0.setBounds(865, 190, 90, 30);
radiobutton0.setForeground(Color.WHITE);
radiobutton0.setFont(font);
buttongroup.add(radiobutton0);
panel.add(radiobutton0);

radiobutton1 = new JRadioButton("Name");
radiobutton1.setBackground(color2);
radiobutton1.setBounds(960, 190, 90, 30);
radiobutton1.setForeground(Color.WHITE);
radiobutton1.setFont(font);
buttongroup.add(radiobutton1);
panel.add(radiobutton1);

// text field implementation
textfield0.setBounds(40, 70, 800, 30);
textfield0.setFont(font);
panel.add(textfield0);

textfield1.setBounds(860, 420, 190, 30);
textfield1.setFont(font);
panel.add(textfield1);
```

```
// labels implementation
label[0] = new JLabel("          Search");
label[0].setBackground(color3);
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(860, 70, 190, 30);
label[0].setFont(font);
panel.add(label[0]);
label[0].addMouseListener(action);

label[1] = new JLabel("      New Search");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(860, 140, 190, 30);
label[1].setFont(font);
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("     Add to Store");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(860, 470, 190, 30);
label[2].setFont(font);
```

```
panel.add(label[2]);
label[2].addMouseListener(action);

label[3] = new JLabel("      Print Report");
label[3].setBackground(color3);
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(860, 520, 190, 30);
label[3].setFont(font);
panel.add(label[3]);
label[3].addMouseListener(action);

label[4] = new JLabel("          Back");
label[4].setBackground(color3);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(860, 570, 190, 30);
label[4].setFont(font);
panel.add(label[4]);
label[4].addMouseListener(action);

label[5] = new JLabel("   X");
label[5].setBackground(color3);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(1030, 0, 50, 35);
```

```
label[5].setFont(font);
panel.add(label[5]);
label[5].addMouseListener(action);

label[6] = new JLabel("  ---");
label[6].setBackground(color3);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
label[6].setBounds(980, 0, 50, 35);
label[6].setFont(font);
panel.add(label[6]);
label[6].addMouseListener(action);

label[7] = new JLabel("  Stock");
label[7].setBackground(color3);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
label[7].setBounds(0, 0, 120, 35);
label[7].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[7]);

label[8] = new JLabel("");
label[8].setBackground(color3);
label[8].setOpaque(true);
label[8].setForeground(Color.WHITE);
label[8].setBounds(120, 0, 860, 35);
```

```
        label[8].setFont(new Font("seirf", Font.BOLD, 25));
        panel.add(label[8]);

        Drag frameDragListener = new Drag(this);
        label[7].addMouseListener(frameDragListener);
        label[7].addMouseMotionListener(frameDragListener);
        label[8].addMouseListener(frameDragListener);
        label[8].addMouseMotionListener(frameDragListener);
    }

    // function to minimize the form
    private void minimize(){
        this.setState(JFrame.ICONIFIED);
    }

    // function to move books from stock to store
    private void move(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
            sql = "UPDATE books SET InStore = (InStore + ?), InStock = (InStock - ?)
WHERE code = ? AND InStock >= ?";
            }
            else if(radiobutton1.isSelected()){
            sql = "UPDATE books SET InStore = (InStore + ?), InStock = (InStock - ?)
WHERE name = ? AND InStock >= ?";
```

```
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield1.getText());
            preparedstatement.setString(2, textfield1.getText());
            preparedstatement.setString(3, textfield0.getText());
            preparedstatement.setString(4, textfield1.getText());
            if (preparedstatement.executeUpdate()>0) {
                JOptionPane.showMessageDialog(null, "Book Transferred to Store
Successfully!");
                clear();
            }else{
                JOptionPane.showMessageDialog(null, "The Entered Amount are not Avilable in
The Stock!");
            }
        } catch (SQLException ex) {
            Logger.getLogger(Stock.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    // back function to return to admin form
    private void back(){
        Admin admin = new Admin();
        admin.setVisible(true);
        this.setVisible(true);
        this.setVisible(false);
        admin.username(label[8].getText());
    }
```

```
    // username function to get the username from login
    public void username(String user){
        label[8].setText(user);
    }

    // clear function to clear all text field after each process
    private void clear(){
        textfield0.setText("");
        textfield1.setText("");
        view();
    }

    // view function to view all books in the database
    private void view(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM books ORDER BY code";
            preparedstatement = connection.prepareStatement(sql);
            resultset = preparedstatement.executeQuery();
            table.setModel(DbUtils.resultSetToTableModel(resultset));
            connection.close();
        }catch(SQLException ex){
            JOptionPane.showMessageDialog(null, "Cannot View!"+ex.getMessage());
        }
```

```
    }

    // search function to search for specific book data in the database
    private void search(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                sql = "SELECT * FROM books WHERE code = ?";
            }
            else if(radiobutton1.isSelected()){
                sql = "SELECT * FROM books WHERE name = ?";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield0.getText());
            resultset = preparedstatement.executeQuery();
            table.setModel(DbUtils.resultSetToTableModel(resultset));
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, "Please Select Search Method");
        }
    }

    // print function to print report with all books data are in the database
    private void print(){
        MessageFormat header = new MessageFormat("Book Report");
```

```
    MessageFormat footer = new MessageFormat("Book Report");
    try{
        table.print(JTable.PrintMode.FIT_WIDTH, header, footer);
    }catch(PrinterException ex){
        JOptionPane.showMessageDialog(null, "Cannot Print!"+ex.getMessage());
    }
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[0]) {
            search();
        }
        if (me.getSource() == label[1]) {
            clear();
        }
        if (me.getSource() == label[2]) {
            move();
        }
        if (me.getSource() == label[3]) {
            print();
        }
        if (me.getSource() == label[4]) {
```

```
            back();
        }
        if (me.getSource() == label[5]) {
            System.exit(0);
        }
        if (me.getSource() == label[6]) {
            minimize();
        }
    }

    @Override
    public void mousePressed(MouseEvent me) {
        if (me.getSource() == label[0]) {
            label[0].setBackground(color1.brighter());
        }
        if (me.getSource() == label[1]) {
            label[1].setBackground(color1.brighter());
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1.brighter());
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color1.brighter());
        }
        if (me.getSource() == label[4]) {
            label[4].setBackground(color1.brighter());
```

```
    }
    if (me.getSource() == label[5]) {
        label[5].setBackground(Color.RED.brighter());
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[0]) {
        label[0].setBackground(color1);
    }
    if (me.getSource() == label[1]) {
        label[1].setBackground(color1);
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1);
    }
```

```
    if (me.getSource() == label[5]) {
        label[5].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1);
    }
}

@Override
public void mouseEntered(MouseEvent me) {
    if (me.getSource() == label[0]) {
        label[0].setBackground(color1);
    }
    if (me.getSource() == label[1]) {
        label[1].setBackground(color1);
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1);
    }
    if (me.getSource() == label[5]) {
```

```
            label[5].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
    }

    @Override
    public void mouseExited(MouseEvent me) {
        if (me.getSource() == label[0]) {
            label[0].setBackground(color3);
        }
        if (me.getSource() == label[1]) {
            label[1].setBackground(color3);
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color3);
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color3);
        }
        if (me.getSource() == label[4]) {
            label[4].setBackground(color3);
        }
        if (me.getSource() == label[5]) {
            label[5].setBackground(color3);
```

```
            }
            if (me.getSource() == label[6]) {
                label[6].setBackground(color3);
            }
        }
    }
}

// Seller Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.awt.print.PrinterException;
import java.sql.*;
import java.text.MessageFormat;
import java.util.logging.*;
import javax.swing.*;
import net.proteanit.sql.DbUtils;

public class Seller extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final JTextField textfield0, textfield1;
    private final JRadioButton radiobutton0, radiobutton1;
```

```
private final ButtonGroup buttongroup;
private final JTable table;
private final JScrollPane scrollPane;
private final Color color1, color2, color3;
private final Font font;
private Connection connection;
private PreparedStatement preparedstatement;
private ResultSet resultset;
private String sql;

// the class constructor
public Seller(){

  // form implementation
  this.setLocation(200, 60);
  this.setUndecorated(true);
  this.setSize(950, 650);
  this.setResizable(false);
  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

  action = new Action();
  panel = new JPanel();
  label = new JLabel[30];
  textfield0 = new JTextField();
  textfield1 = new JTextField();
  buttongroup = new ButtonGroup();
```

```
table = new JTable();
color1 = new Color(60, 60, 60);
color2 = new Color(30, 30, 30);
color3 = new Color(15, 15, 15);
font = new Font("seirf", Font.BOLD, 22);

// function call
view();

// background implementation
panel.setBackground(color2);
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
panel.setLayout(null);
this.add(panel);

// scrollPane implementation
panel.add(table);
scrollPane = new JScrollPane(table);
scrollPane.setBounds(40,140,600,460);
scrollPane.setEnabled(false);
panel.add(scrollPane);

// radio buttons implementation
radiobutton0 = new JRadioButton("Code");
radiobutton0.setBackground(color2);
radiobutton0.setBounds(700, 150, 90, 30);
```

```
radiobutton0.setForeground(Color.WHITE);
radiobutton0.setFont(font);
buttongroup.add(radiobutton0);
panel.add(radiobutton0);

radiobutton1 = new JRadioButton("Name");
radiobutton1.setBackground(color2);
radiobutton1.setBounds(800, 150, 90, 30);
radiobutton1.setForeground(Color.WHITE);
radiobutton1.setFont(font);
buttongroup.add(radiobutton1);
panel.add(radiobutton1);

// text field implementation
textfield0.setBounds(40, 70, 600, 30);
textfield0.setFont(font);
panel.add(textfield0);

textfield1.setBounds(175, 105, 100, 30);
textfield1.setFont(font);
panel.add(textfield1);

// labels implementation
label[0] = new JLabel("              Search");
label[0].setBackground(color3);
label[0].setOpaque(true);
```

```
label[0].setForeground(Color.WHITE);
label[0].setBounds(660, 70, 250, 30);
label[0].setFont(font);
panel.add(label[0]);
label[0].addMouseListener(action);

label[1] = new JLabel("              Clear");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(660, 110, 250, 30);
label[1].setFont(font);
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("           Add Item");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(660, 410, 250, 30);
label[2].setFont(font);
panel.add(label[2]);
label[2].addMouseListener(action);

label[3] = new JLabel("           Print Bill");
label[3].setBackground(color3);
```

```
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(660, 450, 250, 30);
label[3].setFont(font);
panel.add(label[3]);
label[3].addMouseListener(action);

label[4] = new JLabel("              Logout");
label[4].setBackground(color3);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(660, 570, 250, 30);
label[4].setFont(font);
panel.add(label[4]);
label[4].addMouseListener(action);

label[5] = new JLabel("   X");
label[5].setBackground(color3);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(900, 0, 50, 35);
label[5].setFont(font);
panel.add(label[5]);
label[5].addMouseListener(action);

label[6] = new JLabel("  ---");
```

```
label[6].setBackground(color3);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
label[6].setBounds(850, 0, 50, 35);
label[6].setFont(font);
panel.add(label[6]);
label[6].addMouseListener(action);

label[7] = new JLabel("  Seller ");
label[7].setBackground(color3);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
label[7].setBounds(0, 0, 110, 35);
label[7].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[7]);

label[8] = new JLabel("          Delete Item");
label[8].setBackground(color3);
label[8].setOpaque(true);
label[8].setForeground(Color.WHITE);
label[8].setBounds(660, 490, 250, 30);
label[8].setFont(font);
panel.add(label[8]);
label[8].addMouseListener(action);

label[9] = new JLabel("Code: ");
```

```
label[9].setBackground(color2);
label[9].setOpaque(true);
label[9].setForeground(Color.WHITE);
label[9].setBounds(660, 200, 75, 30);
label[9].setFont(font);
panel.add(label[9]);

label[10] = new JLabel("Name: ");
label[10].setBackground(color2);
label[10].setOpaque(true);
label[10].setForeground(Color.WHITE);
label[10].setBounds(660, 240, 75, 30);
label[10].setFont(font);
panel.add(label[10]);

label[11] = new JLabel("Price: ");
label[11].setBackground(color2);
label[11].setOpaque(true);
label[11].setForeground(Color.WHITE);
label[11].setBounds(660, 280, 75, 30);
label[11].setFont(font);
panel.add(label[11]);

label[13] = new JLabel("");
label[13].setBackground(color2);
label[13].setOpaque(true);
```

```
label[13].setForeground(Color.YELLOW);
label[13].setBounds(740, 200, 170, 30);
label[13].setFont(font);
panel.add(label[13]);

label[14] = new JLabel("");
label[14].setBackground(color2);
label[14].setOpaque(true);
label[14].setForeground(Color.YELLOW);
label[14].setBounds(740, 240, 170, 30);
label[14].setFont(font);
panel.add(label[14]);

label[15] = new JLabel("");
label[15].setBackground(color2);
label[15].setOpaque(true);
label[15].setForeground(Color.YELLOW);
label[15].setBounds(740, 280, 170, 30);
label[15].setFont(font);
panel.add(label[15]);

label[16] = new JLabel("Total: ");
label[16].setBackground(color2);
label[16].setOpaque(true);
label[16].setForeground(Color.WHITE);
label[16].setBounds(660, 360, 75, 30);
```

```
label[16].setFont(font);
panel.add(label[16]);

label[17] = new JLabel("");
label[17].setBackground(color2);
label[17].setOpaque(true);
label[17].setForeground(Color.YELLOW);
label[17].setBounds(740, 360, 170, 30);
label[17].setFont(font);
panel.add(label[17]);

label[18] = new JLabel("              Profile");
label[18].setBackground(color3);
label[18].setOpaque(true);
label[18].setForeground(Color.WHITE);
label[18].setBounds(660, 530, 250, 30);
label[18].setFont(font);
panel.add(label[18]);
label[18].addMouseListener(action);

label[19] = new JLabel("Status: ");
label[19].setBackground(color2);
label[19].setOpaque(true);
label[19].setForeground(Color.WHITE);
label[19].setBounds(660, 320, 80, 30);
label[19].setFont(font);
```

```
panel.add(label[19]);


label[20] = new JLabel("");
label[20].setBackground(color2);
label[20].setOpaque(true);
label[20].setForeground(Color.YELLOW);
label[20].setBounds(740, 320, 170, 30);
label[20].setFont(font);
panel.add(label[20]);


label[21] = new JLabel("");
label[21].setBackground(color3);
label[21].setOpaque(true);
label[21].setForeground(Color.WHITE);
label[21].setBounds(110, 0, 750, 35);
label[21].setFont(new Font("seirf", Font.BOLD, 25));
panel.add(label[21]);


label[22] = new JLabel("Bill Number:");
label[22].setBackground(color2);
label[22].setOpaque(true);
label[22].setForeground(Color.WHITE);
label[22].setBounds(40, 105, 130, 30);
label[22].setFont(font);
panel.add(label[22]);
```

```
        Drag frameDragListener = new Drag(this);
        label[7].addMouseListener(frameDragListener);
        label[7].addMouseMotionListener(frameDragListener);
        label[21].addMouseListener(frameDragListener);
        label[21].addMouseMotionListener(frameDragListener);
    }

    // addhistory function to add items to history
    private void addhistory(){
        try{
            connection = DriverManager.getConnection("jdbc:derby://localhost:1527/textbook",
"a", "1234");
            sql = "INSERT INTO history VALUES(?,?,?,?)";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield1.getText());
            preparedstatement.setString(2, label[13].getText());
            preparedstatement.setString(3, label[14].getText());
            preparedstatement.setString(4, label[15].getText());
            if(preparedstatement.executeUpdate()>0){
                clear();
                view();
                total();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
```

```
    }
  }

  // delete function to delete an item from the history
  private void deletehistory(){
    try{
      connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
      if (radiobutton0.isSelected()) {
        sql = "DELETE FROM history WHERE book_code = ?";
      }
      else if(radiobutton1.isSelected()){
        sql = "DELETE FROM history WHERE book_name = ?";
      }
      preparedstatement = connection.prepareStatement(sql);
      preparedstatement.setString(1,textfield0.getText());
      if(preparedstatement.executeUpdate()>0){
        clear();
        view();
        total();
      }
      connection.close();
    }catch(SQLException e){
      JOptionPane.showMessageDialog(null, e);
    }
  }
```

```
    // add function to add items to bill
    private void add(){
        try{
            connection = DriverManager.getConnection("jdbc:derby://localhost:1527/textbook",
"a", "1234");
            sql = "INSERT INTO bills VALUES(?,?,?,?)";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield1.getText());
            preparedstatement.setString(2, label[13].getText());
            preparedstatement.setString(3, label[14].getText());
            preparedstatement.setString(4, label[15].getText());
            if(preparedstatement.executeUpdate()>0){
                move();
                addhistory();
                clear();
                view();
                total();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // delete function to delete an item from the bill
```

```
    private void delete(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                sql = "DELETE FROM bills WHERE book_code = ?";
            }
            else if(radiobutton1.isSelected()){
                sql = "DELETE FROM bills WHERE book_name = ?";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield0.getText());
            if(preparedstatement.executeUpdate()>0){
                returnback();
                deletehistory();
                clear();
                view();
                total();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // function to move books from store to bill
```

```
    private void move(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "UPDATE books SET InStore = (InStore - 1) WHERE code = ? AND InStore
>= 1";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, label[13].getText());
            preparedstatement.executeUpdate();
        } catch (SQLException ex) {
            Logger.getLogger(Stock.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    // function to return books from bill to store
    private void returnback(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "UPDATE books SET InStore = (InStore + 1) WHERE name = ?";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            preparedstatement.executeUpdate();
        } catch (SQLException ex) {
            Logger.getLogger(Stock.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
```

```
// logout function to retun to Home Page
private void logout(){
    this.setVisible(false);
    Home home = new Home();
    home.setVisible(true);
}

// username function to get the username from login
public void username(String user){
    label[21].setText(user);
}

// profile function to go to user profile
private void profile(){
    this.setVisible(false);
    Profile profile = new Profile();
    profile.setVisible(true);
    profile.username(label[21].getText(), "seller");
    profile.userseller();
}

// clear function to clear all text field after each process
private void clear(){
    textfield0.setText("");
    label[13].setText("");
```

```
        label[14].setText("");
        label[15].setText("");
        label[20].setText("");
    }

    // trunc function to delete all record in bills table
    private void trunc(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "TRUNCATE TABLE bills";
            preparedstatement = connection.prepareStatement(sql);
            if(preparedstatement.executeUpdate()>0){
                clear();
                view();
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // print function to print bill
    private void print(){
        MessageFormat header = new MessageFormat("Books Bill");
        MessageFormat footer = new MessageFormat("Total: "+label[17].getText()+"  EGP");
```

```
        try{
            table.print(JTable.PrintMode.FIT_WIDTH, header, footer);
            label[17].setText("");
            trunc();
            view();
        }catch(PrinterException ex){
            JOptionPane.showMessageDialog(null, "Cannot Print!"+ex.getMessage());
        }
    }

    // function to minimize the form
    private void minimize(){
        this.setState(JFrame.ICONIFIED);
    }

    // search function to search for specific book data in the database
    private void search(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
                sql = "SELECT code,name,price FROM books WHERE code = ?";
            }
            else if(radiobutton1.isSelected()){
                sql = "SELECT code,name,price FROM books WHERE name = ?";
            }
```

```
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,textfield0.getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
               label[13].setText(resultset.getString(1));
               label[14].setText(resultset.getString(2));
               label[15].setText(resultset.getString(3));
               available();
            }else{
               JOptionPane.showMessageDialog(null, "NO Book Found!");
               clear();
            }
            connection.close();
         }catch(SQLException e){
            JOptionPane.showMessageDialog(null, "Please Select Search Method");
         }
      }

      // available function to check if a book is available or not in the store
      private void available(){
         try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            if (radiobutton0.isSelected()) {
               sql = "SELECT InStore FROM books WHERE code = ? AND InStore> 0";
            }
```

```
            else if(radiobutton1.isSelected()){
                sql = "SELECT InStore FROM books WHERE name = ? AND InStore> 0";
            }
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1, textfield0.getText());
            resultset = preparedstatement.executeQuery();
            if (resultset.next()) {
                label[20].setText("Available");
            }else{
                label[20].setText("Not Available");
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // view function to view all books that was added to the bill
    private void view(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM bills ORDER BY bill_number";
            preparedstatement = connection.prepareStatement(sql);
            resultset = preparedstatement.executeQuery();
            table.setModel(DbUtils.resultSetToTableModel(resultset));
```

```
            connection.close();
        }catch(SQLException ex){
            JOptionPane.showMessageDialog(null, "Cannot View!"+ex.getMessage());
        }
    }

    // total function to calculate total price of all books in the bill
    private void total(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT SUM(price) FROM bills";
            preparedstatement = connection.prepareStatement(sql);
            resultset = preparedstatement.executeQuery();
            if (resultset.next()) {
                label[17].setText(resultset.getString(1));
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // private claas to contain mouse actions
    private class Action implements MouseListener{
```

```
@Override
public void mouseClicked(MouseEvent me) {
    if (me.getSource() == label[0]) {
        search();
    }
    if (me.getSource() == label[1]) {
        clear();
    }
    if (me.getSource() == label[2]) {
        add();
    }
    if (me.getSource() == label[3]) {
        print();
    }
    if (me.getSource() == label[4]) {
        logout();
    }
    if (me.getSource() == label[5]) {
        trunc();
        System.exit(0);
    }
    if (me.getSource() == label[6]) {
        minimize();
    }
    if (me.getSource() == label[8]) {
        delete();
```

```
    }
    if (me.getSource() == label[18]) {
        profile();
    }
}

@Override
public void mousePressed(MouseEvent me) {
    if (me.getSource() == label[0]) {
        label[0].setBackground(color1.brighter());
    }
    if (me.getSource() == label[1]) {
        label[1].setBackground(color1.brighter());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1.brighter());
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1.brighter());
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1.brighter());
    }
    if (me.getSource() == label[5]) {
        label[5].setBackground(Color.RED.brighter());
    }
```

```
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1.brighter());
    }
    if (me.getSource() == label[8]) {
        label[8].setBackground(color1.brighter());
    }
    if (me.getSource() == label[18]) {
        label[18].setBackground(color1.brighter());
    }
}

@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[0]) {
        label[0].setBackground(color1);
    }
    if (me.getSource() == label[1]) {
        label[1].setBackground(color1);
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
```

```
            label[4].setBackground(color1);
        }
        if (me.getSource() == label[5]) {
            label[5].setBackground(Color.RED.darker());
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color1);
        }
        if (me.getSource() == label[8]) {
            label[8].setBackground(color1);
        }
        if (me.getSource() == label[18]) {
            label[18].setBackground(color1);
        }
    }

    @Override
    public void mouseEntered(MouseEvent me) {
        if (me.getSource() == label[0]) {
            label[0].setBackground(color1);
        }
        if (me.getSource() == label[1]) {
            label[1].setBackground(color1);
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color1);
```

```
    }
    if (me.getSource() == label[3]) {
        label[3].setBackground(color1);
    }
    if (me.getSource() == label[4]) {
        label[4].setBackground(color1);
    }
    if (me.getSource() == label[5]) {
        label[5].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[6]) {
        label[6].setBackground(color1);
    }
    if (me.getSource() == label[8]) {
        label[8].setBackground(color1);
    }
    if (me.getSource() == label[18]) {
        label[18].setBackground(color1);
    }
}

@Override
public void mouseExited(MouseEvent me) {
    if (me.getSource() == label[0]) {
        label[0].setBackground(color3);
    }
```

```
        if (me.getSource() == label[1]) {
            label[1].setBackground(color3);
        }
        if (me.getSource() == label[2]) {
            label[2].setBackground(color3);
        }
        if (me.getSource() == label[3]) {
            label[3].setBackground(color3);
        }
        if (me.getSource() == label[4]) {
            label[4].setBackground(color3);
        }
        if (me.getSource() == label[5]) {
            label[5].setBackground(color3);
        }
        if (me.getSource() == label[6]) {
            label[6].setBackground(color3);
        }
        if (me.getSource() == label[8]) {
            label[8].setBackground(color3);
        }
        if (me.getSource() == label[18]) {
            label[18].setBackground(color3);
        }
    }
}
```

```
}

// Profile Page Class
// imported libraries
import java.awt.*;
import java.awt.event.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.*;

public class Profile extends JFrame{

    private final Action action;
    private final JPanel panel;
    private final JLabel[] label;
    private final Color color1, color2, color3;
    private final Font font;
    private Connection connection;
    private PreparedStatement preparedstatement;
    private ResultSet resultset;
    private String sql;

    // the class constructor
```

```
public Profile(){

    // form implementation
    this.setLocation(400, 100);
    this.setSize(550, 500);
    this.setUndecorated(true);
    this.setResizable(false);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    action = new Action();
    panel = new JPanel();
    label = new JLabel[20];
    color1 = new Color(60, 60, 60);
    color2 = new Color(30, 30, 30);
    color3 = new Color(15, 15, 15);
    font = new Font("seirf", Font.BOLD, 22);

    // background implementation
    panel.setBackground(color2);
    panel.setBorder(BorderFactory.createLineBorder(Color.BLACK));
    panel.setLayout(null);
    this.add(panel);

    // labels implementation
    label[0] = new JLabel("  Profile");
    label[0].setBackground(color3);
```

```
label[0].setOpaque(true);
label[0].setForeground(Color.WHITE);
label[0].setBounds(0, 0, 450, 35);
label[0].setFont(new Font("seirf", Font.BOLD, 30));
panel.add(label[0]);

label[1] = new JLabel("   X");
label[1].setBackground(color3);
label[1].setOpaque(true);
label[1].setForeground(Color.WHITE);
label[1].setBounds(500, 0, 50, 35);
label[1].setFont(font);
panel.add(label[1]);
label[1].addMouseListener(action);

label[2] = new JLabel("  ---");
label[2].setBackground(color3);
label[2].setOpaque(true);
label[2].setForeground(Color.WHITE);
label[2].setBounds(450, 0, 50, 35);
label[2].setFont(font);
panel.add(label[2]);
label[2].addMouseListener(action);

label[3] = new JLabel("  ID:");
label[3].setBackground(color2);
```

```
label[3].setOpaque(true);
label[3].setForeground(Color.WHITE);
label[3].setBounds(50, 100, 150, 30);
label[3].setFont(font);
panel.add(label[3]);

label[4] = new JLabel("  Name:");
label[4].setBackground(color2);
label[4].setOpaque(true);
label[4].setForeground(Color.WHITE);
label[4].setBounds(50, 150, 150, 30);
label[4].setFont(font);
panel.add(label[4]);

label[5] = new JLabel("  UserName:");
label[5].setBackground(color2);
label[5].setOpaque(true);
label[5].setForeground(Color.WHITE);
label[5].setBounds(50, 200, 150, 30);
label[5].setFont(font);
panel.add(label[5]);

label[6] = new JLabel("  Password:");
label[6].setBackground(color2);
label[6].setOpaque(true);
label[6].setForeground(Color.WHITE);
```

```
label[6].setBounds(50, 250, 150, 30);
label[6].setFont(font);
panel.add(label[6]);

label[7] = new JLabel("  Phone:");
label[7].setBackground(color2);
label[7].setOpaque(true);
label[7].setForeground(Color.WHITE);
label[7].setBounds(50, 300, 150, 30);
label[7].setFont(font);
panel.add(label[7]);

label[8] = new JLabel("  Type:");
label[8].setBackground(color2);
label[8].setOpaque(true);
label[8].setForeground(Color.WHITE);
label[8].setBounds(50, 350, 150, 30);
label[8].setFont(font);
panel.add(label[8]);

label[9] = new JLabel("  Salary:");
label[9].setBackground(color2);
label[9].setOpaque(true);
label[9].setForeground(Color.WHITE);
label[9].setBounds(50, 400, 150, 30);
label[9].setFont(font);
```

```
panel.add(label[9]);

label[10] = new JLabel("");
label[10].setBackground(color1);
label[10].setOpaque(true);
label[10].setForeground(Color.WHITE);
label[10].setBounds(210, 100, 250, 30);
label[10].setFont(font);
panel.add(label[10]);

label[11] = new JLabel("");
label[11].setBackground(color1);
label[11].setOpaque(true);
label[11].setForeground(Color.WHITE);
label[11].setBounds(210, 150, 250, 30);
label[11].setFont(font);
panel.add(label[11]);

label[12] = new JLabel("");
label[12].setBackground(color1);
label[12].setOpaque(true);
label[12].setForeground(Color.WHITE);
label[12].setBounds(210, 200, 250, 30);
label[12].setFont(font);
panel.add(label[12]);
```

```
label[13] = new JLabel("");
label[13].setBackground(color1);
label[13].setOpaque(true);
label[13].setForeground(Color.WHITE);
label[13].setBounds(210, 250, 250, 30);
label[13].setFont(font);
panel.add(label[13]);

label[14] = new JLabel("");
label[14].setBackground(color1);
label[14].setOpaque(true);
label[14].setForeground(Color.WHITE);
label[14].setBounds(210, 300, 250, 30);
label[14].setFont(font);
panel.add(label[14]);

label[15] = new JLabel("");
label[15].setBackground(color1);
label[15].setOpaque(true);
label[15].setForeground(Color.WHITE);
label[15].setBounds(210, 350, 250, 30);
label[15].setFont(font);
panel.add(label[15]);

label[16] = new JLabel("");
label[16].setBackground(color1);
```

```
    label[16].setOpaque(true);
    label[16].setForeground(Color.WHITE);
    label[16].setBounds(210, 400, 250, 30);
    label[16].setFont(font);
    panel.add(label[16]);

    label[17] = new JLabel("        Back");
    label[17].setBackground(color3);
    label[17].setOpaque(true);
    label[17].setForeground(Color.WHITE);
    label[17].setBounds(200, 450, 150, 30);
    label[17].setFont(font);
    panel.add(label[17]);
    label[17].addMouseListener(action);

    Drag frameDragListener = new Drag(this);
    label[0].addMouseListener(frameDragListener);
    label[0].addMouseMotionListener(frameDragListener);
}

// username function to get the username from login
void username(String user, String type){
    label[12].setText(user);
    label[15].setText(type);
}
```

```
    // useradmin function to get admin data from database
    public void useradmin(){
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM admin WHERE username = ?";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,label[12].getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
                label[10].setText(resultset.getString(1));
                label[11].setText(resultset.getString(2));
                label[12].setText(resultset.getString(3));
                label[13].setText(resultset.getString(4));
                label[14].setText(resultset.getString(5));
                label[15].setText(resultset.getString(6));
                label[16].setText(resultset.getString(7));
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // userseller function to get seller data from database
    public void userseller(){
```

```
        try{
            connection =
DriverManager.getConnection("jdbc:derby://localhost:1527/textbook", "a", "1234");
            sql = "SELECT * FROM seller WHERE username = ?";
            preparedstatement = connection.prepareStatement(sql);
            preparedstatement.setString(1,label[12].getText());
            resultset = preparedstatement.executeQuery();
            if(resultset.next()){
                label[10].setText(resultset.getString(1));
                label[11].setText(resultset.getString(2));
                label[12].setText(resultset.getString(3));
                label[13].setText(resultset.getString(4));
                label[14].setText(resultset.getString(5));
                label[15].setText(resultset.getString(6));
                label[16].setText(resultset.getString(7));
            }
            connection.close();
        }catch(SQLException e){
            JOptionPane.showMessageDialog(null, e);
        }
    }

    // function to minimize the form
    private void minimize(){
        this.setState(JFrame.ICONIFIED);
    }
```

```
// function to go back to admin form
private void backtoadmin(){
    Admin admin = new Admin();
    admin.setVisible(true);
    this.setVisible(true);
    this.setVisible(false);
    admin.username(label[12].getText());
}

// function to go back to seller form
private void backtoseller(){
    Seller seller = new Seller();
    seller.setVisible(true);
    this.setVisible(true);
    this.setVisible(false);
    seller.username(label[12].getText());
}

// private claas to contain mouse actions
private class Action implements MouseListener{

    @Override
    public void mouseClicked(MouseEvent me) {
        if (me.getSource() == label[1]) {
            System.exit(0);
```

```
    }
    if (me.getSource() == label[2]) {
        minimize();
    }
    if (me.getSource() == label[17]) {
        if (label[15].getText().equals("admin")) {
            backtoadmin();
        }
        else if (label[15].getText().equals("seller")) {
            backtoseller();
        }
    }
}

@Override
public void mousePressed(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.brighter());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1.brighter());
    }
    if (me.getSource() == label[17]) {
        label[17].setBackground(color1.brighter());
    }
}
```

```
@Override
public void mouseReleased(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[17]) {
        label[17].setBackground(color1);
    }
}

@Override
public void mouseEntered(MouseEvent me) {
    if (me.getSource() == label[1]) {
        label[1].setBackground(Color.RED.darker());
    }
    if (me.getSource() == label[2]) {
        label[2].setBackground(color1);
    }
    if (me.getSource() == label[17]) {
        label[17].setBackground(color1);
    }
}
```

```
        @Override
        public void mouseExited(MouseEvent me) {
            if (me.getSource() == label[1]) {
                label[1].setBackground(color3);
            }
            if (me.getSource() == label[2]) {
                label[2].setBackground(color3);
            }
            if (me.getSource() == label[17]) {
                label[17].setBackground(color3);
            }
        }
    }
}
```

References

Sites references:

- https://www.google.com/search?q=Functional+diagram+define&source=lmns&hl=en&sa=X&ved=2ahUKEwitw-W2kvTtAhVRwYUKHUUSBKIQ_AUoAHoECAEQAA
- https://en.wikipedia.org/wiki/Use_case_diagram
- https://en.wikipedia.org/wiki/State_diagram
- https://www.lucidchart.com/pages/templates/data-flow/lucidchart-data-flow-diagram-level-1
- https://www.uml-diagrams.org/class-diagrams-overview.html
- https://www.lucidchart.com/pages/er-diagrams#section_0
- https://www.modernanalyst.com/Careers/InterviewQuestions/tabid/128/ID/1433/What-is-a-Context-Diagram-and-what-are-the-benefits-of-creating-one.aspx
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/
- https://www.praxisframework.org/en/library/activity-on-arrow-diagram#:~:text=In%20an%20activity%2Don%2Darrow,called%20the%20i%2Dnode).&text=A%20network%20diagram%20is%20created,their%20dependence%20upon%20each%20other
- https://www.productplan.com/glossary/pert-chart/
- https://www.lucidchart.com/pages/data-flow-diagram#:~:text=DFD%20Level%200%20is%20also,its%20relationship%20to%20external%20entities

ملخص

تصميم وإنشاء مخزون الكتب النصية ، في هذه الورقة ، قمنا بتصميم وتنفيذ نظام لفحص وإدارة مخزون الكتب بسهولة باستخدام تطبيق سطح المكتب. في الطريقة التقليدية السابقة ، يقوم المستخدم بزيارة متجر الكتب مباشرة ثم يقوم بفحص المخزون. ومع ذلك ، في هذه الحالة ، تكون إمكانية وصول المستخدم غير مريحة ، ومن الصعب التحقق من المخزون أو إعادة تأكيده. من أجل حل هذه المضايقات ، نقدم نظام إدارة مخزون الكتب لطلب شراء الكتاب والاستعلام عن لوحة الإعلانات حسب حقل الكتاب ، ومن ثم توفير فحص المخزون في الوقت الفعلي مع تطبيق الكتاب ووظائف الشراء. بالإضافة إلى ذلك ، من خلال وظيفة الإشعار الفوري ، يمكننا اللحاق بحالة شراء الكتب المطلوبة في تطبيق سطح المكتب في الوقت الفعلي. لذلك ، قمنا بتصميم وتنفيذ وظيفة تمكن مشغلي المكتبات والمستخدمين من الإدارة بسهولة ويسر من خلال توفير معلومات إحصائية عن بيانات المخزون.

Cairo Higher Institute for Engineering, Computer Science and Management

معهد القاهره العالي للهندسه وعلوم الحاسب والاداره

مشروع تحليل وتصميم نظم

Text Book Inventory

مشرف المشروع : م. عبد الرحمن سامى

قائد فريق المشروع: أحمد شوقي

رقم الطالب: ٢٠١٨٠٣٠٠٢٠

عضو رقم ١: بسنت محمد

رقم الطالب: ٢٠١٨٠٣٠٠٤٧

عضو رقم ٢: يوستينا نشأت

رقم الطالب: ٢٠١٨٠٣٠٢٦٩

عضو رقم ٣: عمر علام

رقم الطالب: ٢٠١٨٠٣٠٠٨٦

القاهرة ٢٠٢٠